FinSight: Enterprise Credit Risk Analytics & Automated Loan Underwriting Engine

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

Rajendra	**: 2420030609**
Mallikarjun	**: 2420030615**
Praveen	**: 2420030370**

Under the Esteemed Guidance of

Dr. N. Sunanda

Associate Professor

Department of COMPUTER SCIENCE AND ENGINEERING

AY – 2026-2027

Semester: ODD



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Declaration

The Project Report entitled "**FinSight: Enterprise Credit Risk Analytics & Automated Loan Underwriting Engine**" is a record of Bonafide work of **Rajendra – 2420030609, Mallikarjun – 2420030615, Praveen – 2420030370** submitted in partial fulfillment for the award of B. Tech in **COMPUTER SCIENCE AND ENGINEERING** to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Rajendra – 2420030609

Mallikarjun – 2420030615

Praveen – 2420030370

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project based report entitled "**FinSight: Enterprise Credit Risk Analytics & Automated Loan Underwriting Engine**" is a bonafide work done and submitted by **Rajendra – 2420030609, Mallikarjun – 2420030615, Praveen – 2420030370** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of **COMPUTER SCIENCE AND ENGINEERING**, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in the completion of our project.

Our greatest appreciation goes to our Course Coordinator **[Dr. N. Sunanda]** and our guide **Dr. N. Sunanda**, Department of **COMPUTER SCIENCE AND ENGINEERING**, whose tremendous support, encouragement and guidance cannot be expressed in words and were invaluable throughout this project.

We express our gratitude to **P.VENKATESHWAR RAO**, Head of the Department of **COMPUTER SCIENCE AND ENGINEERING**, for providing us with adequate facilities, ways and means by which we were able to complete this project.

We thank all the members of the teaching and non-teaching staff, and everyone who assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

Rajendra – 2420030609

Mallikarjun – 2420030615

Praveen – 2420030370

Abstract

Lending institutions such as banks, non-banking finance companies and fintech platforms receive large volumes of loan applications that must be evaluated against credit bureau records, income, existing obligations and repayment behaviour. Manual assessment is slow, varies from one underwriter to another, is hard to audit and does not scale, while regulators expect every decision to be consistent and explainable.

FinSight is an enterprise-style credit risk analytics and automated loan underwriting engine built to address this problem. The system converts raw applicant data into engineered risk features (Debt-to-Income ratio, Loan-to-Income ratio, card utilisation, delinquency and inquiry velocity, employment stability and bank-account conduct), estimates a Probability of Default using three model families (an XGBoost-style gradient-boosted champion, a Random Forest challenger and a Logistic Regression baseline), and rescales the result to a 300–900 FinSight credit score with an eight-step credit grade (AAA to D) and a risk-based interest rate. A configurable policy layer of knockout rules, soft flags and score cut-offs then issues an APPROVED, REFERRED or REJECTED decision together with an approved limit and a human-readable reason.

Every decision is accompanied by a SHAP-style factor attribution and a printable audit certificate so that outcomes can be justified to applicants, auditors and regulators. The application is delivered as a React 18 and TypeScript single-page web application with six modules: a portfolio analytics dashboard, a live bank-loan underwriter with KYC and bank-statement intake, a model benchmarking laboratory (ROC, KS and feature importance), a policy rules editor with an impact simulator, a persistent bank loan ledger with batch ingestion, decisioning and CSV export, and an academic showcase. A companion Python pipeline generates a 10,000-record synthetic applicant dataset and reports the benchmark evaluation matrix, in which the champion model reaches an AUC-ROC of 0.884 and a KS statistic of 52.6%. This report documents the requirements, technology, architecture, design, implementation, features, testing, deployment, limitations and future scope of the project.

Keywords: credit risk, probability of default, loan underwriting, explainable AI, SHAP, XGBoost, policy engine, React, TypeScript.

Index

S. No.	Chapters	Topics	Page no.
	Acknowledgement		4
	Abstract		5
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	7-9
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	10-11
3	Technology Stack	3.1 Front-end (React, TypeScript, Tailwind CSS) 3.2 Back-end (service layer and Python pipeline) 3.3 Database (typed in-memory datasets) 3.4 Version Control 3.5 APIs or External services	12-13
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture	14-16
5	Design	5.1 Wireframes or mockups 5.2 ER Diagram / Database schema	17-19
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 Backend logic (scoring, policy engine, explainability) 6.4 Database connectivity 6.5 API integration 6.6 Authentication & Authorization	20-25
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	26-32
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	33-36
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	37
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	38-39
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	40
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	41-42
13	References	Books, papers, documentation sites used	43
14	Appendices	A. Screenshots of the app B. Sample code snippets C. Installation/setup instructions D. User manual or guide E. Certifications	44-50

CHAPTER 1
Introduction

1.1 Background of the Project

Credit is the working capital of an economy. Every home-improvement loan, vehicle loan, business-expansion loan or debt-consolidation loan a lender sanctions carries the risk that the borrower will not repay. Banks, non-banking finance companies (NBFCs) and digital lenders therefore invest heavily in credit risk assessment, the process of estimating how likely an applicant is to default and deciding whether, and on what terms, to lend.

Traditionally this assessment has combined a bureau score (in India, the CIBIL score on a 300–900 scale), manual review of income documents and a loan officer's judgement. As application volumes grow, this approach becomes slow and inconsistent: two underwriters can reach different decisions on the same file, reasons are rarely recorded in a structured way, and the institution cannot easily show a regulator why a particular applicant was approved or declined.

Machine learning offers a more consistent alternative. Models such as gradient-boosted decision trees can learn non-linear interactions between features (for example, the combined effect of a high debt burden and a high card utilisation) and produce a calibrated Probability of Default (PD). However, a model alone is not an underwriting system. Lenders must also apply their own credit policy, such as minimum bureau scores and maximum debt-to-income limits, and they must be able to explain each decision. The Reserve Bank of India's fair-practice expectations and the Basel Committee's Internal Ratings-Based (IRB) approach both emphasise calibrated PD estimates, rating grades and auditability.

FinSight was conceived to bring these elements together in a single, working application: feature engineering, comparative model scoring, a configurable policy engine, explainability and a portfolio-level dashboard.

1.2 Problem Statement

Financial institutions evaluate large volumes of loan applications involving credit bureau data, income and transaction records, and repayment history. Manual credit risk assessment is slow, inconsistent across underwriters, difficult to scale and prone to bias, while lenders must still meet strict regulatory and audit requirements.

The core problem is the lack of a unified, data-driven system that can score applicant risk consistently, explain its decisions and apply lending policy automatically, instead of relying on fragmented spreadsheets, manual checks and disconnected scoring tools. FinSight addresses this

by providing an end-to-end credit risk analytics pipeline and an automated underwriting engine that scores applicants, explains the score and issues a consistent, auditable lending decision.

1.3 Scope of the Project

In scope

- Unsecured retail and small-business loan applications in five purposes: Debt Consolidation, Home Improvement, Business Expansion, Personal / Medical and Auto Purchase.
- Feature engineering for Debt-to-Income (DTI), Loan-to-Income (LTI), credit card utilisation, 24-month delinquency count, hard-inquiry velocity, employment tenure and type, and bank-account conduct (average monthly balance, monthly inflows and inward cheque or ECS bounces).
- Probability of Default estimation using three selectable model families, conversion to a 300–900 score, an eight-level credit grade and a risk-based interest rate.
- A rule-based policy layer with knockout rules, borderline (soft) flags and configurable automatic approval and rejection cut-offs.
- Local explainability through SHAP-style factor attributions and a printable audit certificate for each decision.
- Portfolio analytics, model benchmarking (ROC, KS, feature importance), batch decisioning with CSV export, a browser-persisted applicant ledger and an academic showcase of the project.

Out of scope

- Live integration with credit bureaus, core banking systems or payment gateways.
- Server-side persistent storage, user accounts and role-based access control (applications are kept only in the browser's localStorage).
- Production model retraining, monitoring and deployment on real, labelled lending data.
- Secured lending (mortgages), loss-given-default and exposure-at-default modelling.

1.4 Objective(s)

1. To build a data pipeline that ingests and cleans applicant financial, credit bureau and transaction data.
2. To engineer risk-relevant features such as income stability, debt-to-income ratio and repayment history.
3. To develop and compare credit risk scoring models: Logistic Regression, Random Forest and XGBoost.

4. To design an automated underwriting engine that applies the scoring output and lending policy rules to reach an approve / reject / refer decision.
5. To integrate model explainability (SHAP) so that every decision can be justified for compliance and audit.
6. To build an interactive dashboard for risk analysts to monitor portfolio-level risk and underwriting outcomes.

1.5 Importance of the Application

A wrong credit decision is expensive in both directions. Rejecting a good borrower loses revenue and customer goodwill; approving a risky borrower creates a bad debt. Credit risk and underwriting is therefore one of the highest-stakes applications of data science, and the models used must be both accurate and explainable.

Systems of the FinSight type help lenders approve creditworthy applicants faster, reduce non-performing assets by catching high-risk profiles early, and maintain the audit trail that regulators such as the RBI and the Basel framework expect. In the project's portfolio simulation, decision turnaround is presented as falling from about 48 hours for a manual review to under two seconds for an automated one, and a transparent rule layer lets risk teams change policy without re-engineering the model. For students, the project is also a complete exercise in applying data cleaning, feature engineering, classification modelling and explainability to a real, high-impact financial problem.

1.6 Target Users / Audience

User	How they use FinSight
Credit risk analysts	Monitor portfolio KPIs, grade distribution, default-rate trend and compare model performance.
Loan underwriters	Enter a new application with bank-statement details or search a saved one, then view PD, score, grade, APR, limit and the reasons behind the decision.
Credit policy officers	Tune knockout thresholds and auto-approve / auto-reject cut-offs and preview the portfolio impact.
Operations teams	Review the bank loan ledger, ingest batches of applications, filter results and export decisions as CSV.
Auditors and compliance officers	Inspect the SHAP factor attribution and print the audit certificate for any decision.
Students and evaluators	Walk through the academic showcase and benchmark results of the capstone project.

CHAPTER 2
System Requirements

2.1 Hardware Requirements

FinSight is a browser-based application whose computation (feature engineering, scoring and policy evaluation) runs on the client in milliseconds, so the hardware requirements are modest. The requirements below apply to a developer machine used to build and run the project; end users only need a device capable of running a modern web browser.

Component	Minimum	Recommended
Processor	Dual-core, 2.0 GHz	Quad-core, 2.5 GHz or higher
Memory (RAM)	4 GB	8 GB or more (for faster builds and parallel tools)
Storage	1 GB free (project, dependencies and build output)	2 GB free (SSD)
Display	1366 × 768	1920 × 1080 (the dashboard grid is designed for wide screens; a compact mobile navigation is provided for narrow screens)
Network	Internet for the initial dependency installation and web-font loading	Broadband

Table 2.1 Hardware requirements

2.2 Software Requirements

Software	Requirement	Purpose
Operating system	Windows 10/11, macOS or Linux	Development and execution
Node.js	Version 18 or later (v22 was used for verification)	Runs Vite, the TypeScript compiler and the build scripts
npm	Bundled with Node.js	Installs and locks the front-end dependencies
Web browser	Current Chrome, Edge, Firefox or Safari	Runs the single-page application and prints audit certificates
Python	Version 3.9 or later	Runs the ML evaluation pipeline (ml_pipeline/train_models.py)
Python packages	numpy, pandas, scikit-learn, xgboost, scipy (see requirements.txt)	Data generation and model-training stack for the pipeline
Code editor	Visual Studio Code or any TypeScript-aware editor	Editing the source code

Table 2.2 Software requirements

2.3 Development Tools and Frameworks

The dependency versions below are taken from the project's package.json.

Package	Version	Role in the project
react / react-dom	^18.3.1	Component-based user interface and state management
typescript	^5.7.3	Static typing for applicant, result, rule and metric models (strict mode)
vite	^6.1.0	Development server (port 3000) and production bundler
@vitejs/plugin-react	^4.3.4	React fast-refresh and JSX transform for Vite
tailwindcss, postcss, autoprefixer	^3.4.17, ^8.5.2, ^10.4.20	Utility-first styling and the custom brand palette
recharts	^2.15.0	Bar, line and ROC/KS charts
lucide-react	^0.475.0	Icon set used across the interface
clsx, tailwind-merge	^2.1.1, ^3.0.1	Conditional class-name composition
@types/react, @types/react-dom, @types/node	^18.3.18, ^18.3.5, ^22.13.4	Type definitions

Table 2.3 Front-end development dependencies

CHAPTER 3
Technology Stack

FinSight deliberately keeps its stack small: a typed React front end, a pure-TypeScript decision engine that runs in the browser, and a Python pipeline for dataset generation and benchmark reporting. This section describes each technology and why it was chosen.

3.1 Front-end

- **React 18** – The interface is a tree of function components (Navbar, Dashboard, LiveUnderwriter, ModelComparison, PolicyRulesEditor, BatchProcessor, ProjectShowcase, AuditCertificate, TeamModal). React hooks (useState, useMemo) keep the form, model selection and results synchronised so that a score updates as soon as an input changes.
- **TypeScript 5 (strict)** – All domain objects (Applicant, UnderwritingResult, ShapFactor, PolicyRule, ModelMetric, PortfolioStats) are declared in a single types module. Strict typing catches mismatched fields at build time, which is valuable in a domain where a mis-typed threshold changes lending outcomes.
- **Tailwind CSS 3** – Utility classes and a custom palette (a teal "brand" scale and deep-slate backgrounds) deliver a consistent dark analytical theme without a separate stylesheet per component.
- **Recharts** – Declarative charts for the outcome mix, default-rate trend, ROC curves, KS separation and feature-importance views.
- **lucide-react** – A lightweight icon library for navigation and status indicators.
- **Vite 6** – Instant development server and an optimised static production build.

3.2 Back-end

FinSight has no separate server process. The "back end" is a service layer of pure TypeScript modules that run in the browser and could be moved to a server or serverless function unchanged, because they contain no user-interface code:

Module	Responsibility
services/scoringEngine.ts	DTI and LTI calculation, Probability of Default for three model families (with bank-conduct terms in the champion), score scaling, grade assignment, risk-based APR and the model benchmark table.
services/underwritingPolicy.ts	Default policy rules (five knockouts and two soft flags), auto-approve / auto-reject cut-offs, the decision state machine, limit and tenure calculation.

Module	Responsibility
services/shapExplainer.ts	Per-applicant factor attributions (seven factors, including bank-account conduct) and the global feature-importance ranking.
services/mockData.ts	Seed applicants (four archetypes with bank details plus six more), portfolio statistics, ROC and KS series and project metadata.

Table 3.1 Service-layer modules

A companion **Python 3** pipeline (ml_pipeline/train_models.py) uses NumPy to generate a 10,000-record synthetic applicant dataset, engineer the same features and print the benchmark evaluation matrix. Its requirements file lists NumPy, pandas, scikit-learn, XGBoost and SciPy as the model-training stack.

3.3 Database

No database server is used in the current version. Seed applicants, portfolio statistics and benchmark series are typed constants in the data module. The applicant ledger is held in React state in App.tsx and mirrored to the browser's localStorage (key finsight_applicants_ledger), so saved applications persist across reloads in the same browser; results are recalculated on demand. This keeps the prototype self-contained and instantly runnable. Chapter 11 describes how a relational database (for example PostgreSQL) would store applications, decisions and policy versions in a production deployment, using the entity model in Section 5.2 as its starting schema.

3.4 Version Control

The project is organised as a standard Node.js repository (package.json with a lockfile, source in src/, the Python pipeline in ml_pipeline/, and the build output in dist/). It is suitable for Git-based version control with a hosting service such as GitHub, with the lockfile ensuring that every team member and every build resolves identical dependency versions.

3.5 APIs or External Services

- **Google Fonts** – the Plus Jakarta Sans and JetBrains Mono typefaces are loaded from Google Fonts in index.html; the page falls back to system fonts if they are unavailable.
- **Browser APIs** – the Blob and object-URL APIs generate the CSV download of batch decisions, window.print() prints the audit certificate, and localStorage persists the applicant ledger.
- **Bureau and banking APIs** – the batch pipeline stage is labelled "Bureau & Banking API", but in this prototype the data are simulated; integration with real bureau and account-aggregator services is future work.

CHAPTER 4
System Architecture

4.1 High-Level Architecture Diagram

FinSight follows a three-layer architecture. The presentation layer renders the six modules; the decision layer holds all business logic; and the data layer supplies typed datasets and the offline ML pipeline. Dependencies point downwards only: components call services, and services depend on types and data, never on components.

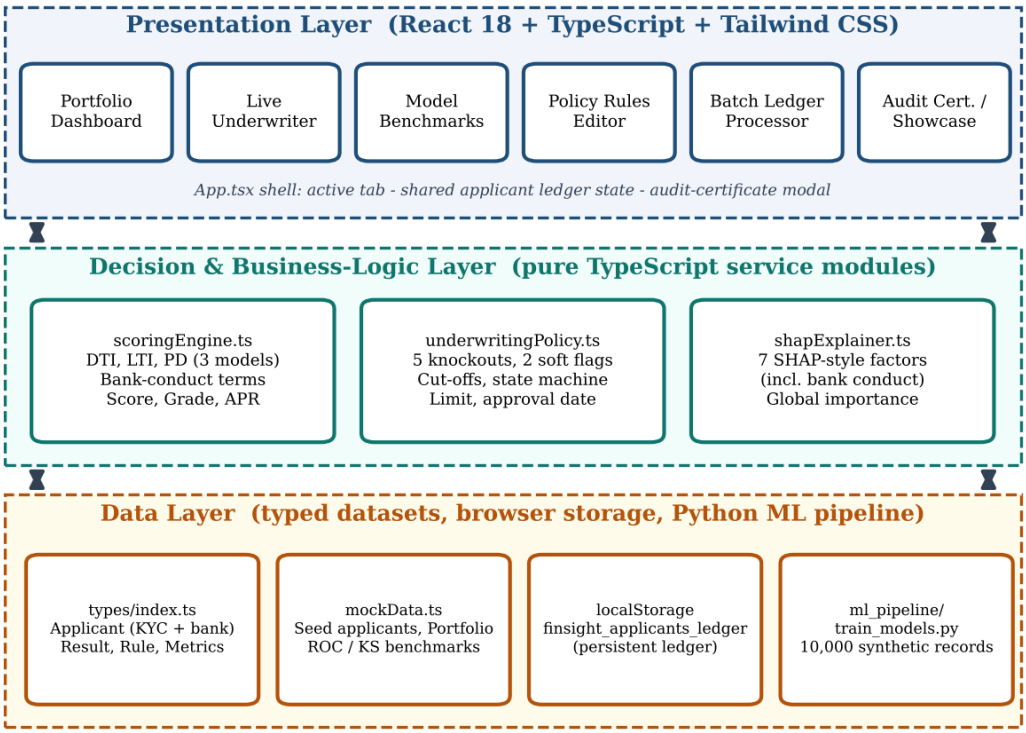


Fig. 4.1 High-level architecture of FinSight

4.2 Description of Each Layer

Presentation layer (front end)

App.tsx is the application shell. It keeps four pieces of state: the active tab, whether the team modal is open, the currently selected decision for the audit certificate and the shared applicant ledger (initialised from localStorage and written back on every change). It renders the navigation bar, the active module and the shared footer and modals. Each module is an independent component that receives only the callbacks it needs (for example, the Live Underwriter and Batch Processor receive a function that opens the audit certificate for a result).

Decision layer (back end logic)

The decision layer is the heart of the system. Given an Applicant, the scoring engine engineers features and estimates a PD; the policy module applies knockout and flag rules and the PD and

score cut-offs to reach a decision; and the explainer produces factor attributions. All functions are deterministic and side-effect free, which makes them straightforward to test (Chapter 8) and to relocate to a server.

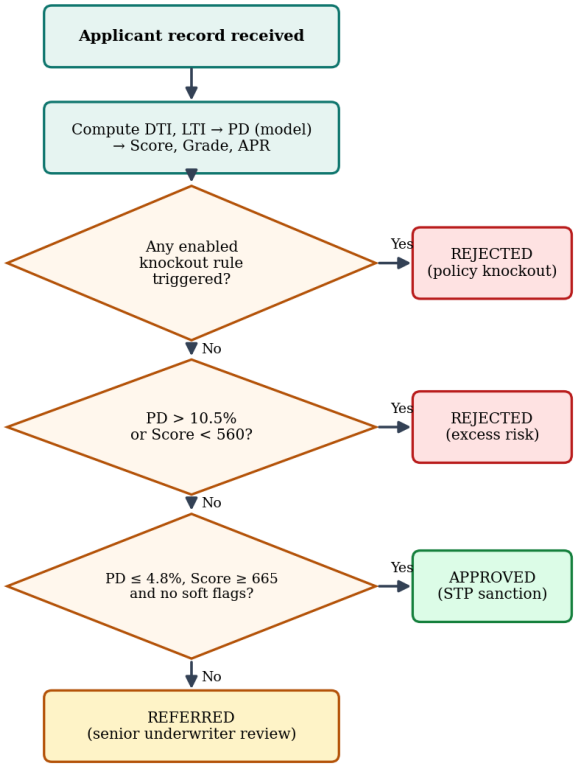


Fig. 4.2 Underwriting decision flow (order of policy evaluation)

Data layer

The types module defines the entities shared by every layer. The data module provides four applicant archetypes with bank-account details, a ten-applicant seed batch, simulated portfolio statistics (14,850 applicants, ₹48.2 crore disbursed, average score 612, approval rate 68%, default rate 3.1%) and the ROC and KS series used by the charts. The Python pipeline generates a synthetic dataset and prints the benchmark matrix. The shared applicant ledger is mirrored to the browser's localStorage so that saved applications survive a reload.

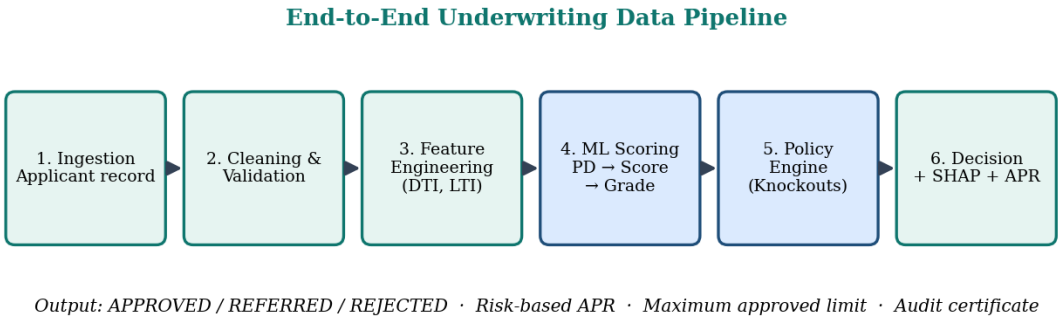


Fig. 4.3 End-to-end underwriting data pipeline

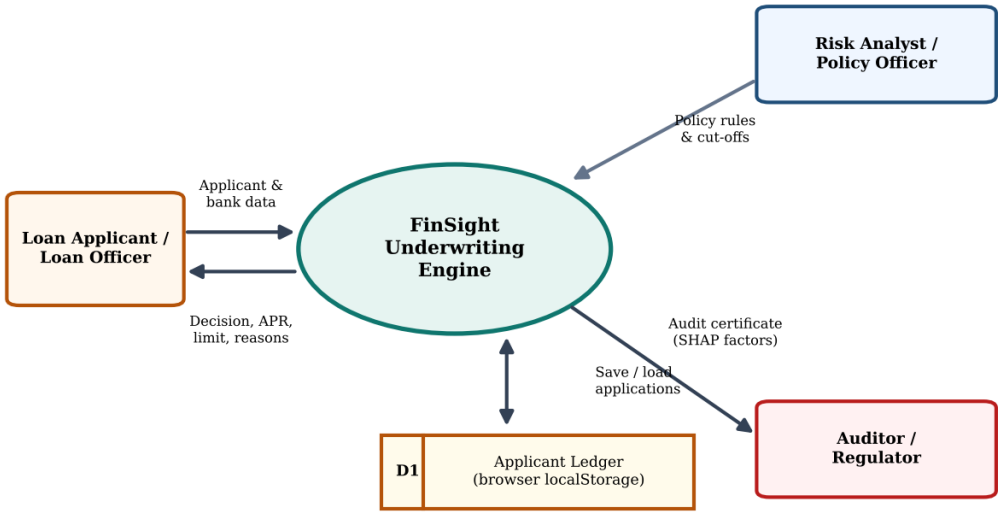


Fig. 4.4 Data-flow diagram (Level 0) of FinSight

4.3 Deployment Architecture

Because the application is a static single-page app, deployment is simple: the TypeScript is type-checked and compiled, Vite bundles the HTML, JavaScript and CSS into the dist/ folder, and any static host can serve it. During development, the Vite dev server runs on port 3000. A CI pipeline (for example GitHub Actions) can run the build on every push and publish dist/ to a static host such as Netlify, Vercel, GitHub Pages or an Nginx server. No server-side runtime, database or secret is required.

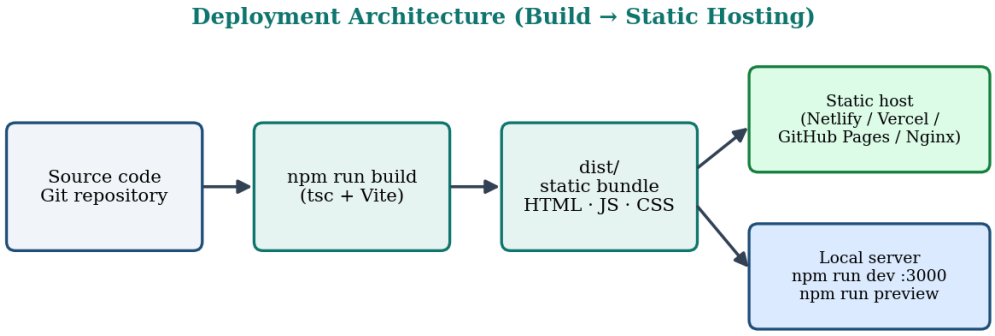


Fig. 4.5 Deployment architecture (build to static hosting)

CHAPTER 5
Design

5.1 Wireframes or Mockups

The interface follows one repeated page pattern so that analysts can move between modules without re-learning the layout: a sticky navigation bar, a banner that states the module's purpose and primary actions, a row of key figures, and a two-panel working area with inputs or charts on the left and results or tables on the right. A footer shows the project guide and opens the team details.

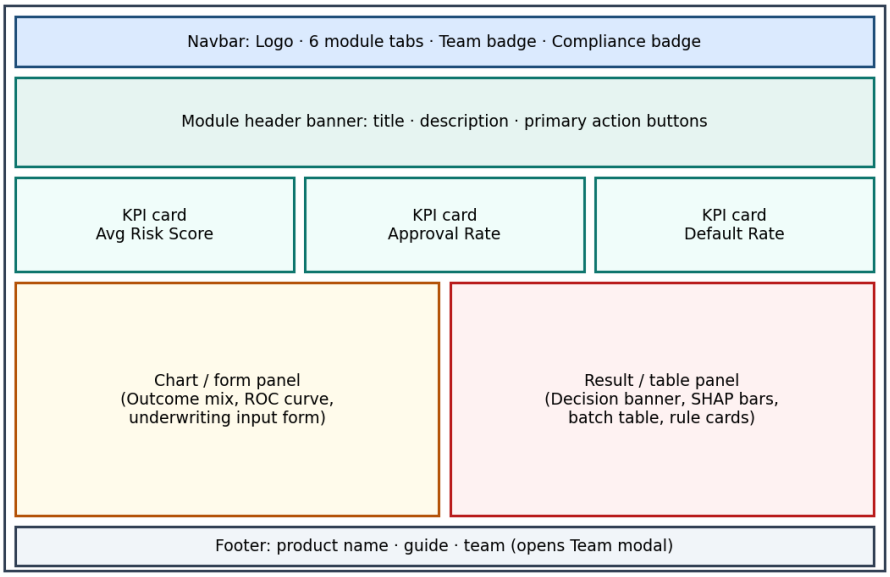


Fig. 5.1 Wireframe of the common page layout

Visual design language

- **Theme** – a dark, low-glare analytical theme (deep slate-teal backgrounds, #0d1f23 to #1c3d44) suited to long monitoring sessions.
- **Accent colour** – a teal brand scale (#14b8a6 as the primary accent) for actions and highlights.
- **Semantic colours** – green for APPROVED, amber for REFERRED and red for REJECTED, repeated consistently in banners, badges, charts and tables.
- **Typography** – Plus Jakarta Sans for text and JetBrains Mono for figures and identifiers, with Inter and system fonts as fallbacks.
- **Responsiveness** – the navigation collapses into a horizontally scrolling bar below the large breakpoint, and the grids reflow from several columns to one.

The working mock-ups (screenshots of the implemented interface) for each module appear in Chapter 7 and Appendix A.

5.2 ER Diagram / Database Schema

The prototype stores data in typed objects rather than tables, but the types form a clear entity model that maps directly to a relational schema. An Applicant produces one or more UnderwritingResults (one per evaluation, optionally with a different model); each result owns a set of ShapFactors; PolicyRules are evaluated for every result; and ModelMetrics describe the models that can be selected.

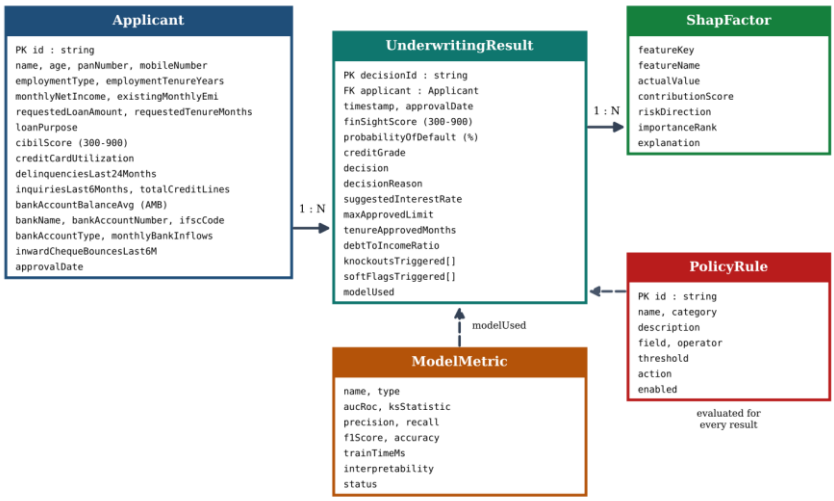


Fig. 5.2 Entity-relationship (class) diagram

Entity	Key attributes	Notes
Applicant	id, name, age, employmentType, employmentTenureYears, monthlyNetIncome, existingMonthlyEmi, requestedLoanAmount, requestedTenureMonths, loanPurpose, cibilScore, creditCardUtilization, delinquenciesLast24Months, inquiriesLast6Months, totalCreditLines, bankAccountBalanceAvg, bankName, bankAccountNumber, ifscCode, bankAccountType, monthlyBankInflows, inwardChequeBouncesLast6M, panNumber, mobileNumber, applicationDate, approvalDate	CIBIL score range 300–900; utilisation 0–100%; employmentType is one of four categories; loanPurpose one of five; bank, PAN and mobile fields are optional.
UnderwritingResult	decisionId, applicant, timestamp, approvalDate, finSightScore, probabilityOfDefault, creditGrade, decision, decisionReason, suggestedInterestRate, maxApprovedLimit, tenureApprovedMonths, debtToIncomeRatio, knockoutsTriggered[], softFlagsTriggered[], shapFactors[], modelUsed	decision is APPROVED, REFERRED or REJECTED; creditGrade is AAA…D; modelUsed is one of three models.
ShapFactor	featureKey, featureName, actualValue, contributionScore, riskDirection, importanceRank, explanation	contributionScore is in score points (positive raises the score).
PolicyRule	id, name, category, description, field, operator, threshold, action, enabled	category is KNOCKOUT, RISK_TIER or BORDERLINE_FLAG; operator is <, >, <=, >= or ==.
ModelMetric	name, type, aucRoc, ksStatistic, precision, recall, f1Score, accuracy, trainTimeMs, interpretability, status	status is Champion, Challenger or Baseline.

Entity	Key attributes	Notes
PortfolioStats	avgRiskScore, approvalRate, defaultRate, totalApplicants, totalDisbursedAmount, outcomesMix, defaultRateTrend[], riskGradeDistribution[]	Aggregate view used by the dashboard.

Table 5.1 Entities and attributes (from types/index.ts)

CHAPTER 6
Implementation

6.1 Module-wise Implementation

Module (tab)	Component file	What it implements
Portfolio Analytics	Dashboard.tsx	KPI cards, decision outcome mix, quarterly default-rate trend (with quarter and loan-type filters), credit-grade distribution and delinquency ageing buckets.
Live Underwriter	LiveUnderwriter.tsx	Blank bank-loan intake form (KYC, bank-statement, loan and bureau sections), search of saved applications, save to ledger, model selector, instant decision banner and the top SHAP factor contributions.
Model Benchmarks	ModelComparison.tsx	Metric cards for the three models and ROC, KS and feature-importance views.
Policy Rules	PolicyRulesEditor.tsx	Toggles and threshold sliders for knockout and flag rules, cut-off sliders and a portfolio impact simulator.
Batch Ingestion	BatchProcessor.tsx	Bank ledger: five-stage pipeline tracker, decision table of all saved applicants (ten seed records) with search and filters, sample ingestion and CSV export.
Academic Showcase	ProjectShowcase.tsx	A 16-slide walkthrough of the project (problem, objectives, literature, architecture, results, conclusion).
Audit Certificate	AuditCertificate.tsx	Modal printable certificate of a decision with factor attribution.
Team details	TeamModal.tsx	Modal with the project guide, team members and roles.

Table 6.1 Modules and their implementing components

6.2 Front-end Logic (UI Rendering, State Management)

State is kept as locally as possible using React hooks, with no external state library. App.tsx owns only cross-cutting state: activeTab selects which module is rendered; isTeamModalOpen controls the team modal; selectedAuditResult holds the decision shown in the audit certificate; and applicants is the shared ledger, loaded from localStorage at start-up and saved back by a useEffect whenever it changes. Modules that need to open the certificate receive a callback prop.

The Live Underwriter keeps the applicant (initialised as a blank application with sensible defaults) and the selected model in state; a search box loads any saved applicant into the form, and the Save to Bank Ledger button passes the applicant up to App.tsx. The decision is not stored; it is derived with useMemo, which re-runs evaluateUnderwriting only when the applicant or the model changes:

```
const result: UnderwritingResult = useMemo(() => {
  return evaluateUnderwriting(applicant, DEFAULT_POLICY_RULES, DEFAULT_CUTOFFS, selectedModel);
}, [applicant, selectedModel]);
```

Deriving results from inputs in this way guarantees that what is displayed is always consistent with the inputs, and it makes every slider movement produce an immediate new decision. The Batch Processor applies the same idea to a list: results are mapped from the applicant array with useMemo, and a second useMemo applies the text search and decision filter. The Policy Rules Editor keeps the rule list and cut-offs in state, and toggling a rule or moving a threshold immutably updates the list.

6.3 Backend Logic (Scoring, Policy Engine, Explainability)

6.3.1 Feature engineering

Debt-to-Income ratio (DTI) includes the estimated instalment of the requested loan. The new EMI is estimated with the reducing-balance formula at a fixed 12% annual rate (1% per month) over the requested tenure n:

```
EMI_new = P × r × (1 + r)^n / ((1 + r)^n − 1),   r = 0.01
DTI (%) = (existing monthly EMI + EMI_new) / max(monthly net income, 1000) × 100
```

Loan-to-Income (LTI) is the requested amount divided by annual income (floored at ₹10,000). Other inputs are standardised around reference points: bureau score centred at 650 (scale 100), DTI at 35% (scale 15), card utilisation at 30% (scale 25) and employment tenure at 4 years (scale 3). Employment type adds a fixed adjustment to the log-odds: −0.55 for government salaried, −0.25 for corporate salaried, +0.15 for self-employed and +0.40 for gig or contractual workers. In the champion model, banking conduct adds 0.45 to the log-odds for every inward cheque or ECS bounce in the last six months, plus a liquidity term based on the ratio of average monthly balance to the requested loan: -0.30 when the ratio exceeds 0.35 and +0.20 when it is below 0.05.

6.3.2 Probability of Default for three model families

The PD is the output of a logistic (sigmoid) transform of a model-specific log-odds, p = 1 / (1 + e^−logit), clamped between 0.3% and 88%. Three variants are implemented so that users can compare model behaviour live:

Model	Log-odds structure (summary)
Logistic Regression (baseline)	Linear: −2.8 − 0.85·bureau + 0.65·DTI + 0.55·utilisation + 0.90·delinquencies + 0.35·inquiries − 0.30·tenure + employment adjustment.
Random Forest (challenger)	Threshold-based: starts at −2.9 and adds step terms when the bureau score is below 620, DTI above 45%, utilisation above 70%, delinquencies ≥ 2, inquiries ≥ 3, with a reduction for tenure above 5 years and 0.8 × the employment adjustment.
XGBoost-style (champion)	Gradient-boosting-style additive terms: −3.1 − 1.05·bureau + 0.75·DTI + 0.65·utilisation + 0.38·inquiries − 0.42·tenure + 0.30·LTI, a delinquency term (+1.6 for two or more, +0.6 for one, −0.4 for none), an interaction penalty of +0.75 when DTI > 45% and utilisation > 65%, a super-prime discount of −0.8 when the bureau score > 780 and DTI < 30%, plus the employment adjustment, +0.45 per inward cheque bounce and the liquidity term.

Table 6.2 Model variants implemented in scoringEngine.ts

The champion's interaction and super-prime terms are what allow it to capture non-linear effects that the linear baseline cannot. The banking-conduct terms (cheque bounces and liquidity) are used only by the champion; the Random Forest and Logistic Regression variants ignore them. These log-odds functions are calibrated approximations that mimic the behaviour of the three model families inside the browser; Section 10.3 discusses this limitation and the path to replacing them with trained models.

```
export function pdToFinSightScore(pdPercent: number): number {
  const pd = pdPercent / 100;
  // FICO/CIBIL standard calibration: Score = 600 + (25 / ln(2)) * ln((1 - pd) / pd)
  const odds = (1 - pd) / Math.max(pd, 0.001);
  const factor = 36.067; // 25 / ln(2)
  const baseScore = 600;
  const score = Math.round(baseScore + factor * Math.log(odds) - 15);
  return Math.min(Math.max(score, 300), 900);
}
```

Listing 6.1 Conversion of PD to the FinSight score (scoringEngine.ts)

6.3.3 Score, grade and pricing

The score follows the standard log-odds scaling used by bureau scores: score = 600 + (25 / ln 2) × ln((1 − PD) / PD) − 15, bounded to 300–900. The PD is also mapped to a credit grade, and the grade determines a spread over a base lending rate of 8.5%:

Grade	PD range	Spread over base	Suggested APR
AAA	< 1.5%	1.75%	10.25%
AA	1.5% – < 3.0%	2.50%	11.00%
A	3.0% – < 5.0%	3.50%	12.00%
BBB	5.0% – < 8.0%	4.75%	13.25%
BB	8.0% – < 12.0%	6.50%	15.00%
B	12.0% – < 18.0%	8.50%	17.00%
CCC	18.0% – < 28.0%	11.00%	19.50%
D	≥ 28.0%	14.50%	23.00%

Table 6.3 Grade, spread and APR mapping

6.3.4 Policy engine and decision state machine

Seven default rules are defined: five knockouts that cause automatic rejection (including the cheque / ECS bounce cap) and two borderline flags that force manual review. Each rule is a data record (field, operator, threshold, action, enabled), so it can be switched off or re-tuned without changing code.

Rule ID	Name	Category	Condition	Action
KO_BUREAU	Minimum Bureau Cutoff	Knockout	cibilScore < 580	REJECTED
KO_DTI	Max Debt-to-Income	Knockout	dti > 58.0	REJECTED

Rule ID	Name	Category	Condition	Action
KO_DELINQ	Delinquency Cap (24 Mo)	Knockout	delinquenciesLast24Months > 2	REJECTED
KO_MIN_INCOME	Minimum Monthly Income	Knockout	monthlyNetIncome < 20,000	REJECTED
KO_BOUNCES	Cheque / ECS Bounces Cap (6 Mo)	Knockout	inwardChequeBouncesLast6M >= 2	REJECTED
FLAG_INQUIRIES	High Velocity Bureau Inquiries	Borderline flag	inquiriesLast6Months ≥ 4	REFERRED
FLAG_UTILIZATION	High Card Utilization Warning	Borderline flag	creditCardUtilization > 80.0	REFERRED

Table 6.4 Default policy rules (underwritingPolicy.ts)

After the rules are evaluated, the decision is taken in a fixed order of precedence (Fig. 4.2): (1) any enabled knockout gives REJECTED with the first knockout as the reason; (2) otherwise a PD above 10.5% or a score below 560 gives REJECTED for excess credit risk; (3) otherwise a PD at or below 4.8%, a score of at least 665 and no soft flags gives APPROVED (straight-through processing); (4) everything else is REFERRED to a senior underwriter, either because a flag fired or because the applicant sits in the near-prime band between the cut-offs.

The approved limit depends on the outcome: zero when rejected; for referrals a conservative counter-offer of the smaller of 75% of the request and 12 months of income; and for approvals the requested amount, increased to 115% (rounded to the nearest ₹10,000) for AAA and AA grades.

```
  // Underwriting Decision State Machine
  let decision: DecisionOutcome = 'REFERRED';
  let decisionReason = '';

  if (knockoutsTriggered.length > 0) {
    decision = 'REJECTED';
    decisionReason = `Policy Knockout: Fails mandatory underwriting baseline
(${knockoutsTriggered[0]}).`;
  } else if (pd > cutoffs.autoRejectMinPd || finSightScore < 560) {
    decision = 'REJECTED';
    decisionReason = `Excessive Credit Risk: Probability of Default (${pd}%) exceeds policy
ceiling (${cutoffs.autoRejectMinPd}%).`;
  } else if (pd <= cutoffs.autoApproveMaxPd && finSightScore >= cutoffs.autoApproveMinScore &&
softFlagsTriggered.length === 0) {
    decision = 'APPROVED';
    decisionReason = `Automated STP Approval: Low estimated default risk (${pd}%) with FinSight
Score ${finSightScore} (${creditGrade}). All underwriting criteria met.`;
  } else {
    decision = 'REFERRED';
    if (softFlagsTriggered.length > 0) {
      decisionReason = `Refer to Senior Underwriter: ${softFlagsTriggered.join('; ')}.`;
    } else {
      decisionReason = `Near-Prime Borderline Review: Probability of Default (${pd}%) sits in
manual discretion band (${cutoffs.autoApproveMaxPd}% - ${cutoffs.autoRejectMinPd}%).`;
    }
  }
```

Listing 6.2 Decision state machine (underwritingPolicy.ts)

6.3.5 Explainability

The explainer decomposes each decision into seven factors: bureau score, DTI, 24-month delinquencies, card utilisation, employment stability, inquiry velocity and bank-account conduct. Each factor receives a signed point contribution (positive raises the score and lowers risk), a risk direction, a rank and a plain-language explanation, and the list is sorted by absolute impact. For example, zero delinquencies add 45 points whereas two or more subtract 90; DTI up to 30% adds 35 points whereas a DTI above 50% subtracts 65. Bank account conduct adds 25 points for zero cheque bounces with an average balance above ₹40,000, and subtracts 40 for one bounce and 80 for two or more. A global importance table (bureau score 32.4%, DTI 24.1%, delinquencies 18.6%, utilisation 11.5%, tenure 7.2%, inquiries 4.2% and LTI 2.0%) supports the model benchmark view.

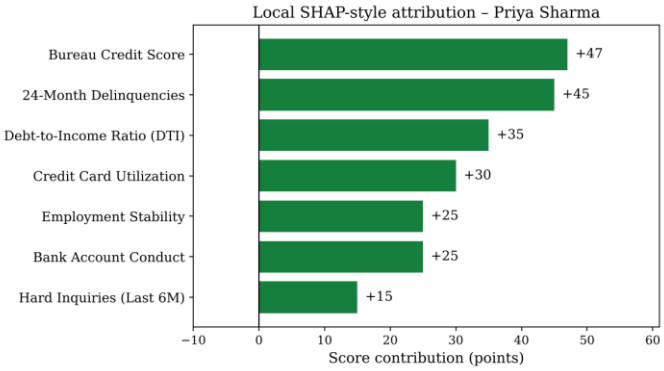


Fig. 6.1 Factor attribution for an approved applicant (Priya Sharma)

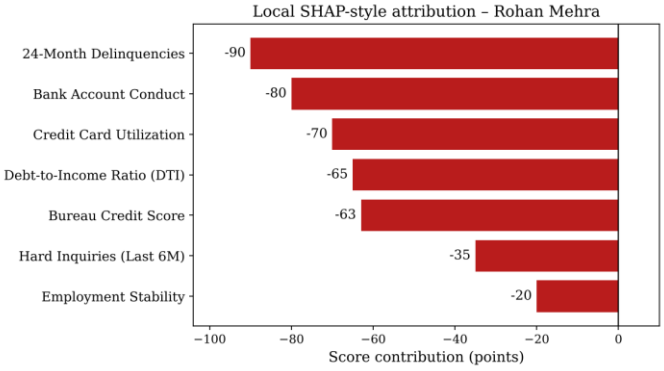


Fig. 6.2 Factor attribution for a rejected applicant (Rohan Mehra)

6.3.6 ML evaluation pipeline (Python)

The Python pipeline performs five reported steps: (1) generate 10,000 synthetic applicants with a fixed random seed (credit scores from a normal distribution with mean 680, log-normal incomes, beta-distributed utilisation, Poisson delinquency and inquiry counts); (2) engineer DTI and LTI; (3) draw a default label from a latent logistic risk function; (4) print the evaluation matrix of the three models; and (5) print the approved / referred / rejected mix. The champion XGBoost model reports AUC-ROC 0.884, KS 52.6%, precision 0.842, recall 0.817 and F1

0.829, against 0.832 / 46.1% / 0.795 / 0.771 / 0.783 for Random Forest and 0.768 / 38.4% / 0.731 / 0.704 / 0.717 for Logistic Regression.

6.4 Database Connectivity

There is no database server in the current version. The data module exports typed constants (seed applicants with bank details, portfolio statistics and curve data) that play the role of a read-only data store. The working data store is the applicant ledger: App.tsx loads it from the browser's localStorage (key finsight_applicants_ledger) at start-up, falls back to the ten seed applicants when nothing is saved, and writes the list back whenever it changes. The Live Underwriter adds or replaces an applicant through a save callback, and the Batch Processor adds two sample applicants when the ingest action is used. Because every component reads data through typed structures, replacing the ledger with calls to a repository or REST client would not change the components.

6.5 API Integration

No external REST API is called. The internal "API" is the set of exported TypeScript functions in the service layer, for example evaluateUnderwriting(applicant, rules, cutoffs, model), calculateDti, calculatePd, pdToFinSightScore, getCreditGrade, calculateRiskBasedInterestRate and calculateShapFactors. The batch module's pipeline tracker names the integration points a production system would use (bureau and banking APIs for ingestion), and CSV export is implemented with the browser Blob API.

6.6 Authentication & Authorization

Authentication and authorisation are not implemented in the prototype; every module is open to the user of the page. Accountability is instead addressed at the decision level: each result receives a unique decision identifier and timestamp, and the audit certificate presents the decision, the policy findings and the factor attribution in a printable form together with a verification fingerprint panel. In a production deployment, role-based access (analyst, underwriter, policy officer, auditor), single sign-on and cryptographically signed decision records would be added (see Chapter 11).

CHAPTER 7
Features

7.1 List of Main Features

#	Feature	Summary
1	Portfolio risk dashboard	Average score, approval rate, default rate, outcome mix, default trend, grade distribution, delinquency ageing.
2	Live loan underwriter	Bank-style intake (KYC, bank-statement, loan and bureau sections), search, real-time scoring and model switching.
3	Multi-model PD scoring	Logistic Regression, Random Forest and XGBoost-style PD side by side.
4	Risk-based pricing	Score, credit grade and APR derived from the PD.
5	Automated policy decisioning	Knockouts, soft flags, cut-offs, approved limit and reason.
6	Explainable AI	Per-decision factor attribution and global feature importance.
7	Model benchmarking lab	AUC, KS, precision, recall, F1, ROC and KS curves.
8	Policy rules editor	Enable, disable and tune rules and cut-offs with an impact preview.
9	Bank loan ledger and batch decisions	Persistent ledger with pipeline tracker, search, filter, sample ingestion and CSV export.
10	Audit certificate	Printable decision certificate for audit and regulatory review.
11	Academic showcase and team details	A 16-slide walkthrough and a modal with guide and team.
12	Banking-footprint scoring	Cheque-bounce knockout, bank-conduct factor and liquidity term in the champion model.

Table 7.1 Main features

7.2 How Each Feature Works

7.2.1 Portfolio risk dashboard

User perspective: the landing page shows three headline figures (average risk score 612, approval rate 68%, default rate 3.1%), a bar chart of the decision outcome mix (68% approved, 17% referred, 15% rejected), a quarterly default-rate line that falls from 4.2% in Q1 to 3.1% in Q4, a credit-grade distribution across 14,850 active accounts and an ageing table from current to 90+ days past due. Quarter and loan-type filters change the view, and buttons jump to the Live Underwriter and Model Benchmarks.

Technical description: the data come from the PortfolioStats object, and the charts are Recharts BarChart and LineChart components. The grade rows show the monotone relationship between grade and observed default rate (0.4% for AAA rising to 26.5% for CCC/D), which is the pattern a well-calibrated rating scale should show.

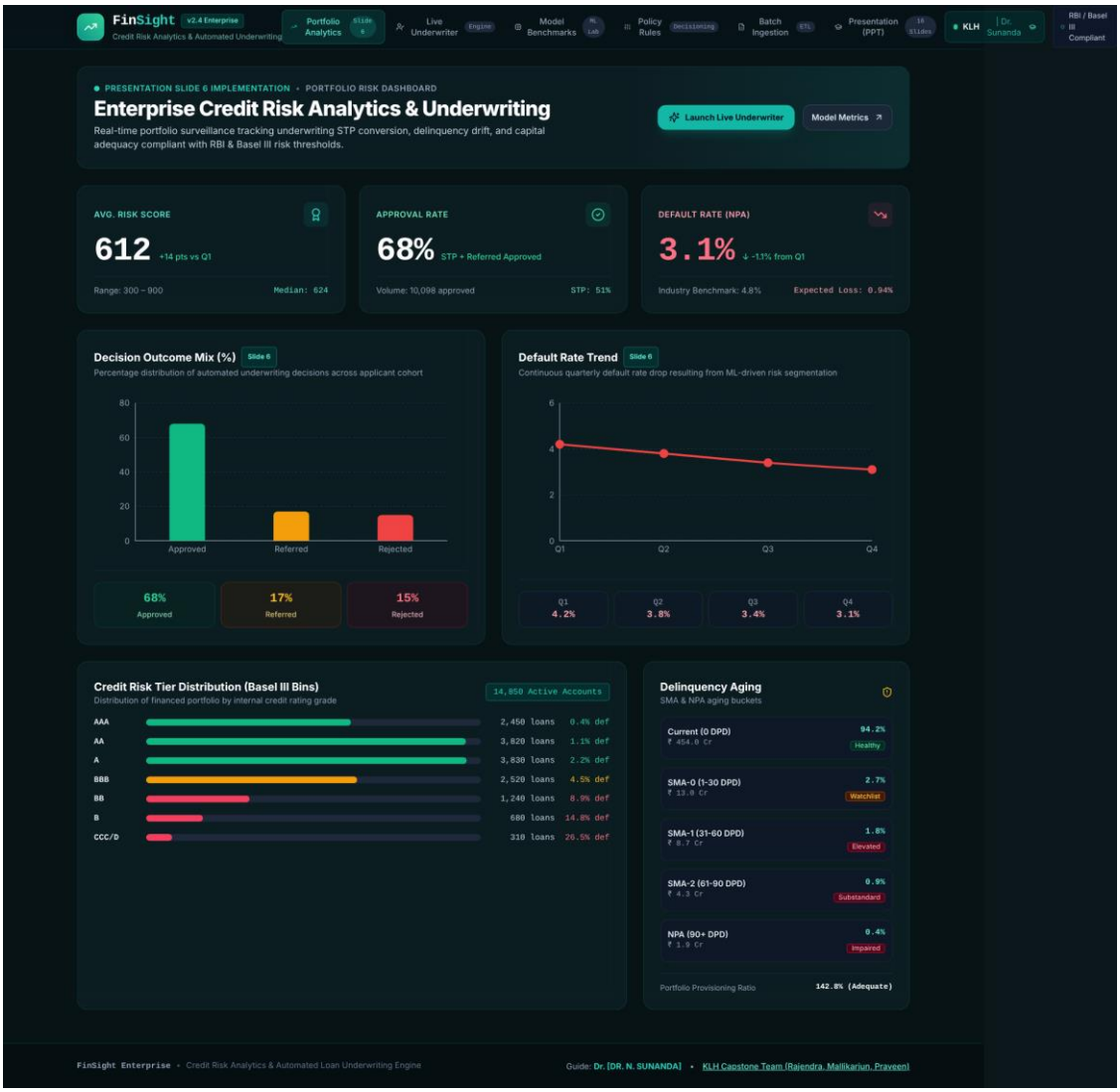


Fig. 7.1 Portfolio risk dashboard



Fig. 7.2 Portfolio by credit grade: volume and default rate

7.2.2 Live loan underwriter

User perspective: the underwriter starts from a blank application (or searches a saved applicant by name, application ID, PAN or bank account) and completes four sections: borrower KYC

and employment (name, PAN, mobile, employment type, income, tenure); bank account and statement details (bank, account type, account number, IFSC, average monthly balance, verified monthly inflows and inward cheque bounces in six months); loan terms (amount, tenure, purpose); and bureau and liabilities (CIBIL score, existing EMI, card utilisation, delinquencies, inquiries, credit lines). A cash-flow variance warning appears when verified inflows fall below 75% of the stated income. A model selector switches between XGBoost (champion), Random Forest and Logistic Regression. The right-hand panel immediately shows the decision banner with the reason and decision date, the FinSight score and grade, the PD, the suggested APR, the approved limit and the four largest score drivers; buttons open the audit certificate or save the application to the bank ledger.

Technical description: inputs are held in state, the full evaluation is derived with useMemo, and DTI and LTI are recalculated for display. The search filters the shared ledger on name, ID, PAN and account number, and saving calls a callback that inserts or replaces the applicant in the ledger held in App.tsx (persisted to localStorage). Every control change re-runs the whole pipeline in the browser without any network call.

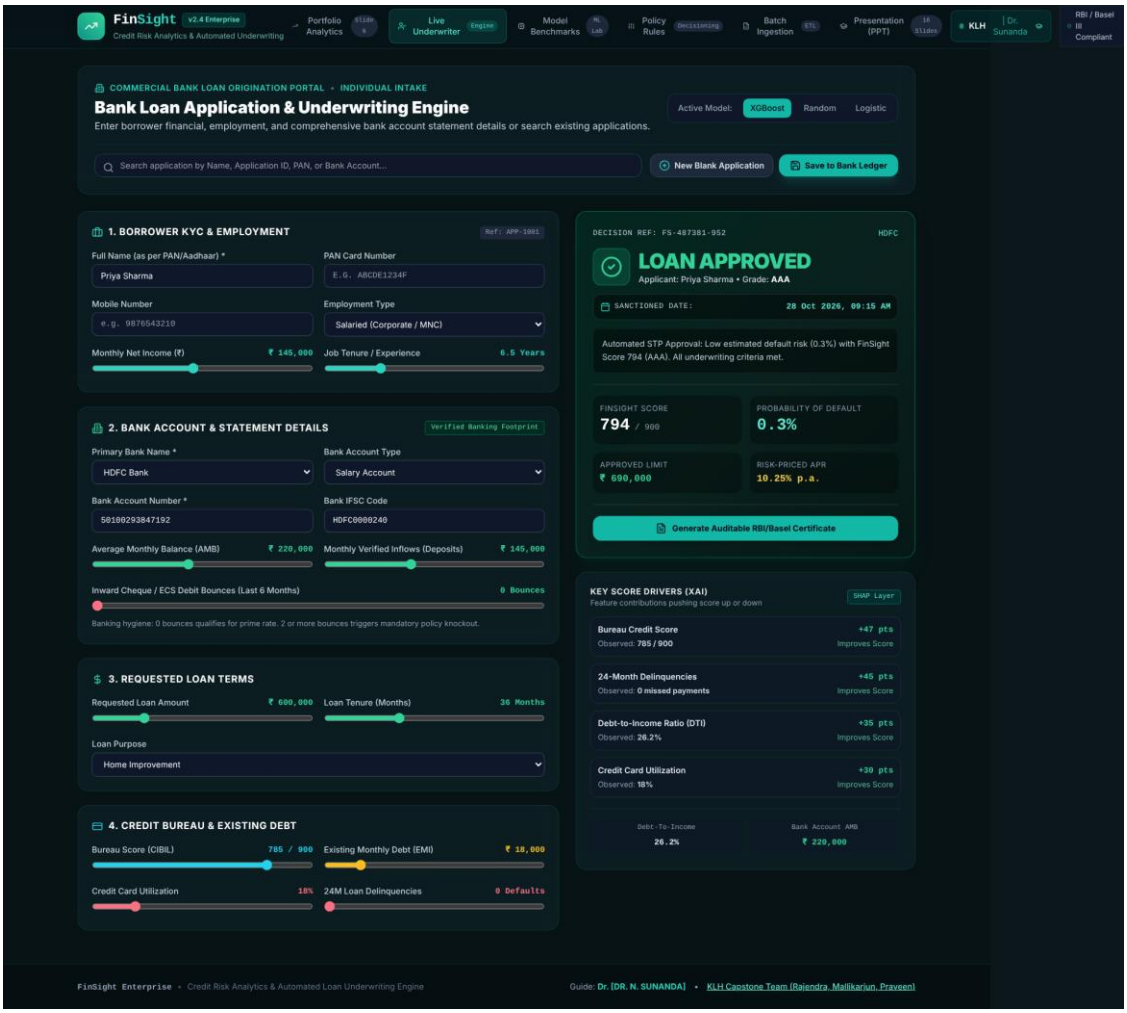


Fig. 7.3 Live underwriter: approved prime applicant

7.2.3 Model benchmarking laboratory

User perspective: three cards present each model's role (champion, challenger, baseline) with AUC-ROC, KS, precision, recall and F1. Tabs switch between ROC curves, KS separation and global feature importance. The champion's AUC of 0.884 and KS of 52.6% exceed the Random Forest (0.832, 46.1%) and the Logistic Regression baseline (0.768, 38.4%); the maximum KS separation occurs at the third risk decile.

Technical description: the cards read the model benchmark table and the charts read the ROC, KS-separation and global feature-importance datasets, rendered with Recharts line and bar charts.

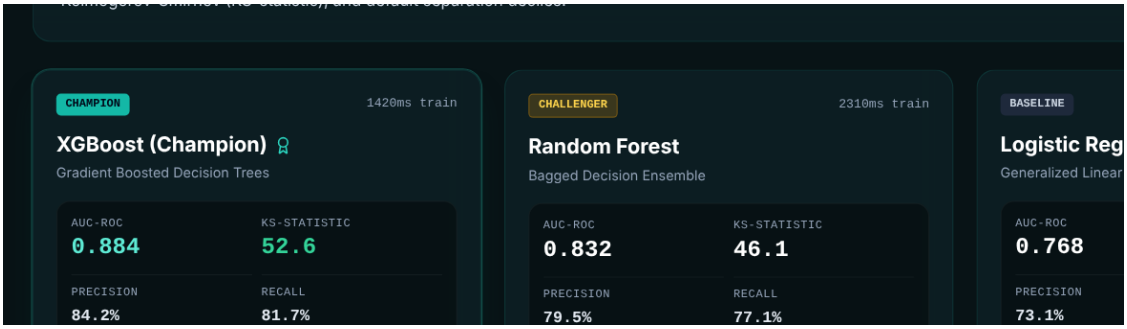


Fig. 7.4 Model benchmark cards

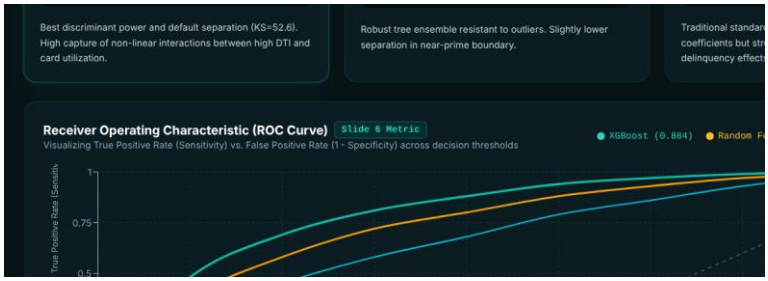


Fig. 7.5 ROC curves of the three models



Fig. 7.6 KS separation of the champion model

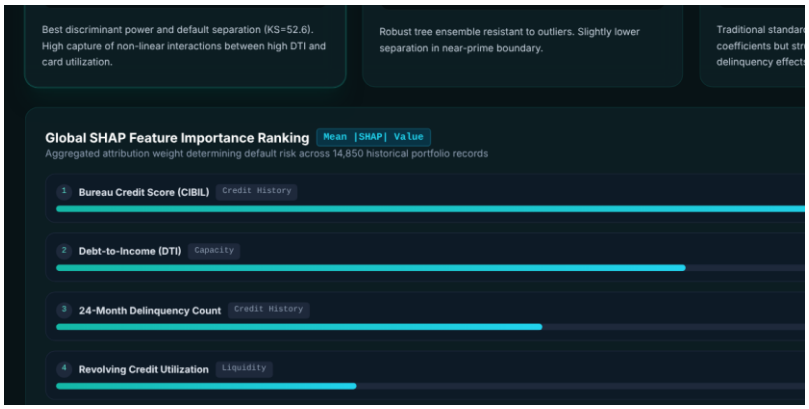


Fig. 7.7 Global feature importance

7.2.4 Policy rules editor

User perspective: a risk officer sees each rule as a card with an on/off toggle and a threshold slider, plus three cut-off sliders: the maximum PD for auto-approval, the minimum score for auto-approval and the PD above which an application is auto-rejected. A simulator panel immediately shows the expected approval rate, default rate and STP rate. Buttons reset to defaults or "publish" the policy.

Technical description: rules and cut-offs are React state arrays updated immutably. The simulator is a transparent heuristic: it moves the 68% baseline approval rate and the 3.1% baseline default rate according to how far the PD cut-off and the DTI knockout deviate from their defaults, bounded to plausible ranges. It illustrates the trade-off between volume and risk rather than re-scoring a portfolio (see Section 10.3).

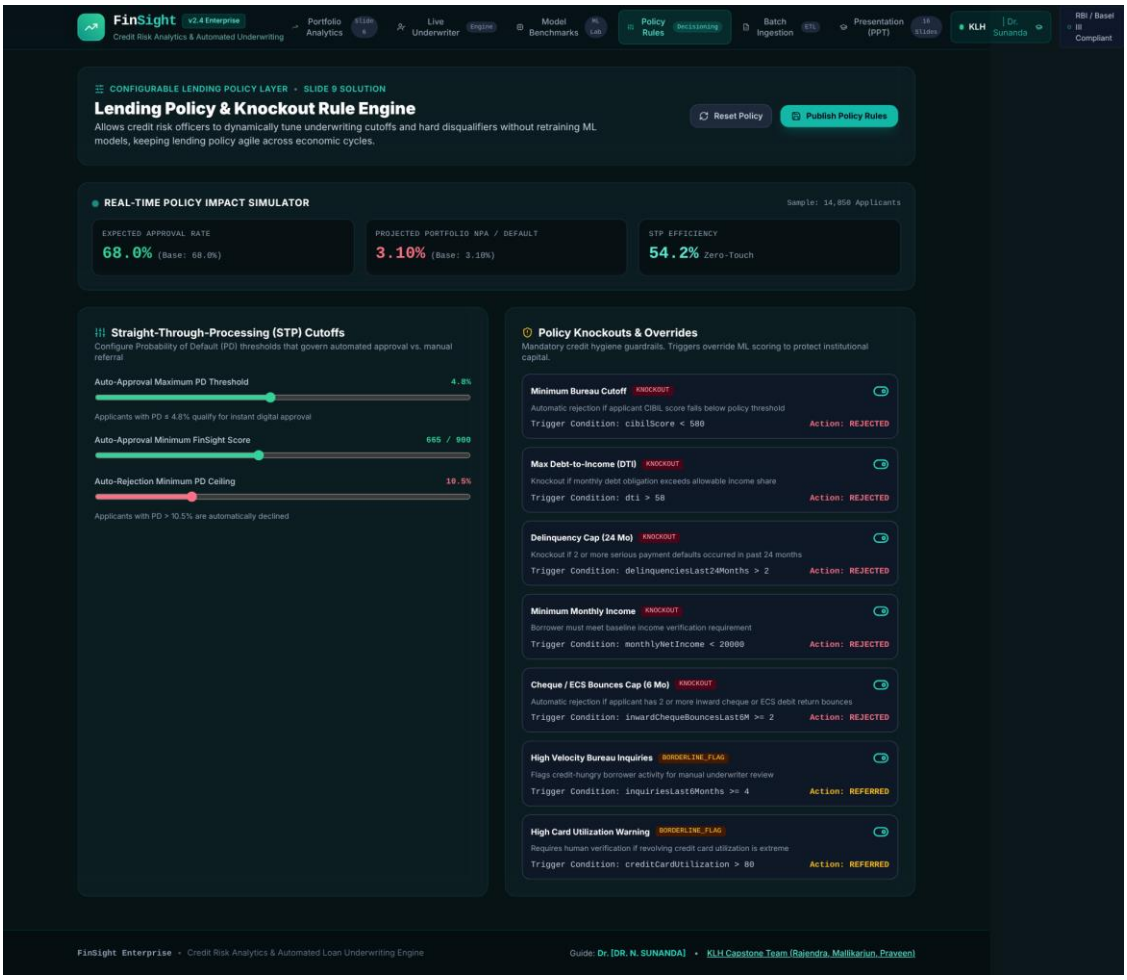


Fig. 7.8 Policy rules editor with impact simulator

7.2.5 Batch ingestion and decision processor

User perspective: a five-stage tracker (ingestion, cleaning, engineering, ML scoring, policy ledger) explains the pipeline. The ledger table lists every saved applicant (ten seed records initially) with applicant, bank and average balance, approval or decision date, requested amount, bureau score, DTI, FinSight score, PD, decision and a certificate button. The user can search by name, ID, bank, account number or date, filter by decision, ingest a sample batch of two applicants, open any row's audit certificate and export the filtered ledger as a CSV file.

Technical description: the list is mapped through evaluateUnderwriting inside useMemo; a second memo filters it. The ingest action appends two sample applicants after a one-second delay, and export builds a CSV string (including bank name, account number, IFSC, balance and cheque bounces), wraps it in a Blob and triggers a download named FinSight_Bank_Ledger_<date>.csv. The list is the shared ledger from App.tsx, so applications saved in the Live Underwriter appear here and survive a reload.

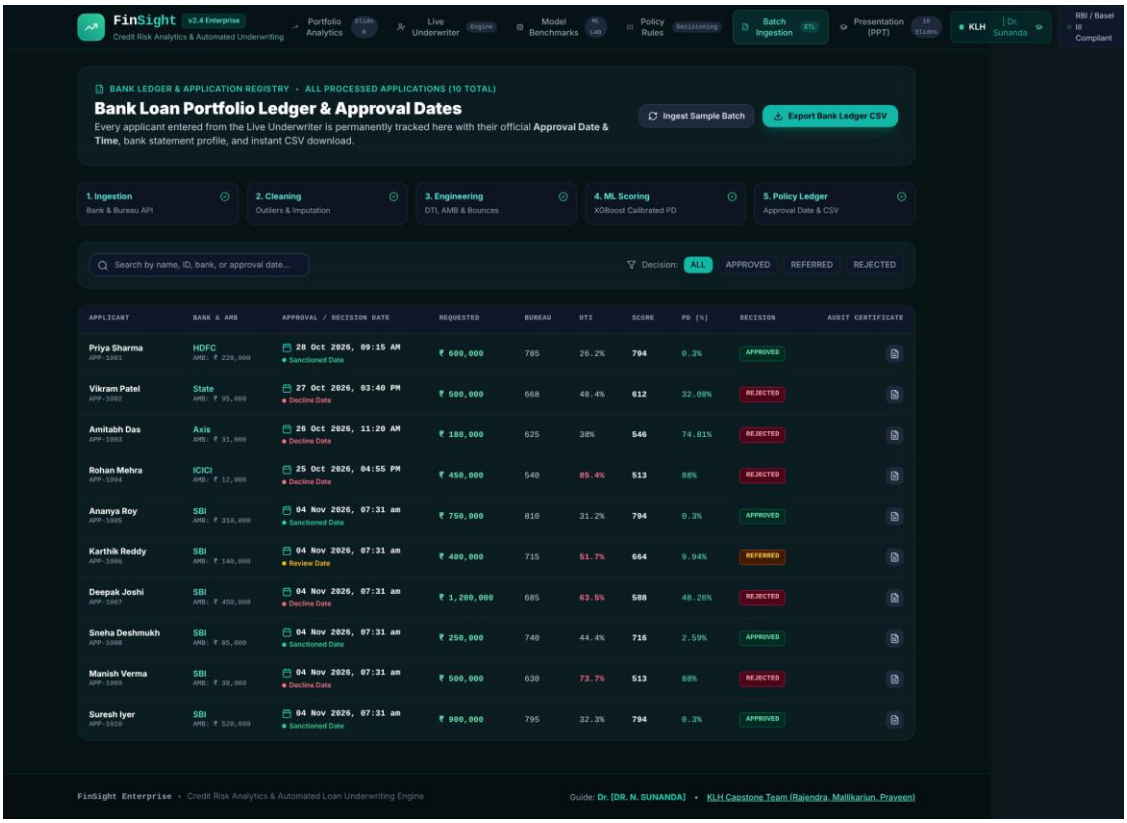


Fig. 7.9 Batch ingestion and decision table

7.2.6 Audit certificate

From the live underwriter or the batch table, the user can open a certificate that records the decision identifier, timestamp, applicant summary, primary bank details, loan terms, score, grade, PD, official decision, the policy finding (for example the specific knockout and its threshold), the factor attribution with explanations and a verification panel. A print control produces a hard copy or a PDF through the browser's print dialog. Appendix A shows an example.

7.2.7 Academic showcase and team details

The showcase presents the project as 16 navigable slides (title, problem statement, relevance, motivation, objectives, expected outcomes, literature survey, gaps, how FinSight closes them, architecture, benchmarking, policy engine, explainability, impact and compliance, conclusion and thanks), and the team modal lists the guide, the three team members, their roles and a shortcut to the showcase.

CHAPTER 8
Testing

8.1 Postman Testing (Frontend / Backend)

Postman is a tool for exercising HTTP endpoints. FinSight currently exposes no HTTP API: the decision engine is a set of TypeScript functions called directly by the React components, and there is no server to send requests to. Postman testing is therefore not applicable to this version of the project, and it was replaced by direct, function-level testing of the service layer (Section 8.4), which exercises exactly the logic that a future REST endpoint would wrap. When the engine is exposed as an API (Chapter 11), the same cases can be turned into a Postman collection: a POST request carrying an applicant JSON document, with assertions on the returned decision, PD, grade and limit.

8.2 Integration Testing

Integration testing focused on the chain that matters most: applicant data → feature engineering → PD → score and grade → policy rules → decision → limit → explanation. A test harness ran the project's own TypeScript service modules (unchanged) under Node.js, so the results below come from the real engine, not from a re-implementation. The harness ran all ten batch applicants through evaluateUnderwriting with the default rules and cut-offs, and a further set of boundary and configuration cases.

On the interface side, the built application was served locally and loaded in headless Chromium, and each module (dashboard, live underwriter, model benchmarks, policy rules, batch processor and showcase) was opened in turn, with a saved applicant loaded through the search box, the audit certificate opened and a new application saved to the ledger and reloaded, to confirm that every tab renders and that the screenshots in this report reflect the working application. The TypeScript compiler runs in strict mode as part of npm run build, so type mismatches between components and the shared types fail the build.

8.3 Tools Used for Testing

Tool	Use
Node.js 22 (native TypeScript execution)	Runs the service modules directly to execute the function-level and integration test cases.
Playwright with headless Chromium	Loads the built application, switches modules, selects presets and captures screenshots.
TypeScript compiler (strict mode)	Static verification of types during the production build.
Python 3 / NumPy	Executes the ML evaluation pipeline and its synthetic data generation.

Tool	Use
Browser developer tools	Inspecting layout, console output and the print view of the audit certificate.

Table 8.1 Testing tools

8.4 Test Cases and Results

8.4.1 End-to-end results for the ten batch applicants

Table 8.2 lists the output of the champion engine for the ten applicants of the batch module under the default policy (knockouts on; auto-approve at PD ≤ 4.8% and score ≥ 665; auto-reject at PD > 10.5%).

ID	Applicant	CIBIL	DTI	PD	Score	Grade	Decision	Max limit
APP-1001	Priya Sharma	785	26.2%	0.3%	794	AAA	**APPROVED**	₹6,90,000
APP-1002	Vikram Patel	668	48.4%	32.08%	612	D	**REJECTED**	0
APP-1003	Amitabh Das	625	38%	74.81%	546	D	**REJECTED**	0
APP-1004	Rohan Mehra	540	85.4%	88%	513	D	**REJECTED**	0
APP-1005	Ananya Roy	810	31.2%	0.3%	794	AAA	**APPROVED**	₹8,60,000
APP-1006	Karthik Reddy	715	51.7%	9.94%	664	BB	**REFERRED**	₹3,00,000
APP-1007	Deepak Joshi	685	63.5%	48.26%	588	D	**REJECTED**	0
APP-1008	Sneha Deshmukh	740	44.4%	2.59%	716	AA	**APPROVED**	₹2,90,000
APP-1009	Manish Verma	630	73.7%	88%	513	D	**REJECTED**	0
APP-1010	Suresh Iyer	795	32.3%	0.3%	794	AAA	**APPROVED**	₹10,30,000

Table 8.2 Engine output for the batch applicants (default policy, XGBoost champion)

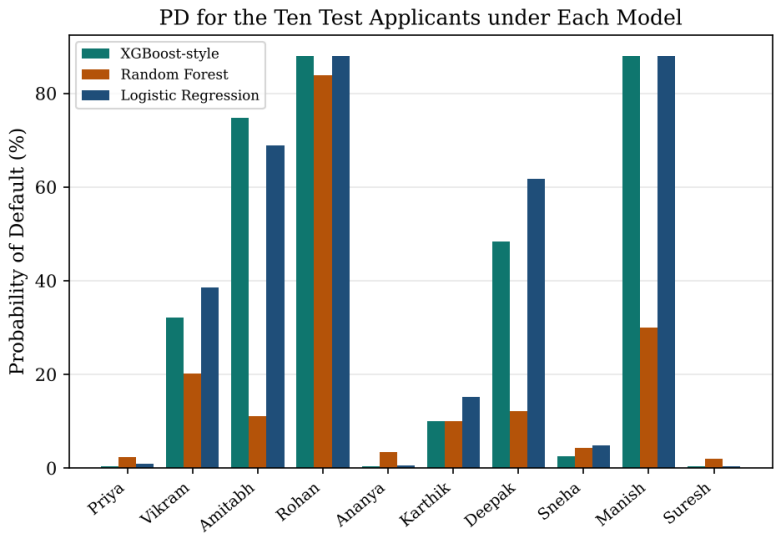


Fig. 8.1 PD of the ten test applicants under the three models

Four applicants (APP-1001, 1005, 1008 and 1010, the prime, government-salaried and long-tenure profiles) were approved with grades AAA or AA; the AAA and AA approvals received a limit of 115% of the request, as designed. One applicant (APP-1006, DTI 51.7%, PD 9.94%)

landed in the near-prime band and was referred with a counter-offer of ₹3,00,000 (75% of the request). Five applicants were rejected; APP-1004 (Rohan Mehra) triggered four knockouts (bureau score 540, DTI 85.4%, three delinquencies and three cheque bounces), and APP-1007 triggered one (DTI 63.5%).

8.4.2 Functional, boundary and configuration test cases

ID	Test case	Expected	Actual	Result
TC-01	Prime applicant (preset 1) default engine	APPROVED	APPROVED	**Pass**
TC-02	CIBIL = 579 (below 580 knockout)	REJECTED	REJECTED / KO=1	**Pass**
TC-03	CIBIL = 580 (boundary, no bureau knockout)	0	0	**Pass**
TC-04	Monthly income = 19,999 (< 20,000 knockout)	REJECTED	REJECTED	**Pass**
TC-05	Delinquencies = 2 (rule is > 2)	0	0	**Pass**
TC-06	Delinquencies = 3 (> 2)	REJECTED	REJECTED	**Pass**
TC-07	Inquiries = 4 with otherwise prime profile (soft flag)	REFERRED	REFERRED	**Pass**
TC-08	Card utilization = 81% (> 80 soft flag)	REFERRED	REFERRED	**Pass**
TC-09	All six rules disabled; prime profile with CIBIL set to 540	Observation: decision then rests on PD / score cut-offs only	APPROVED (PD 2.89%, score 712, grade AA)	**Noted**
TC-10	Disable KO_BUREAU only, CIBIL = 570, prime otherwise	0	0	**Pass**
TC-11	Rejected case returns zero limit and zero tenure	0 / 0	0 / 0	**Pass**
TC-12	AAA approved: limit = 115% of request (rounded to 10,000)	690000	690000	**Pass**
TC-13	Referred case: limit = min(75% of request, annual income)	450000	450000	**Pass**
TC-14	Score mapping: PD 0.3% -> score	794	794	**Pass**
TC-15	Grade mapping: PD 4.99% -> A, PD 5.0% -> BBB	A / BBB	A / BBB	**Pass**
TC-16	APR for grade BBB = base 8.5 + 4.75	13.25	13.25	**Pass**
TC-17	PD lower-bound clamp (very strong applicant)	0.3	0.3	**Pass**
TC-18	PD upper-bound clamp (very weak applicant)	88	88	**Pass**
TC-19	Model switch changes PD (XGB vs RF vs LR) for preset 2	differs	differs (32.08/20.1/38.5)	**Pass**
TC-20	DTI includes estimated new EMI (12% p.a.) for preset 1	26.2	26.2	**Pass**
TC-21	SHAP factors returned sorted by |impact| (7 factors)	7	7	**Pass**
TC-22	Bounces = 1, otherwise prime (below the knockout)	APPROVED; bank factor -40	APPROVED, KO=0, bank factor -40	**Pass**
TC-23	Bounces = 2 (knockout is >= 2)	REJECTED	REJECTED / KO=1	**Pass**
TC-24	Disable KO_BOUNCES only; bounces = 2, otherwise prime	APPROVED	APPROVED / KO=0	**Pass**

ID	Test case	Expected	Actual	Result
TC-25	Preset 2 PD at 0 / 1 / 2 bounces (champion)	PD rises with bounces	32.08 / 42.56 / 53.74	**Pass**
TC-26	Preset 2 PD at balance 3,00,000 / 95,000 / 10,000	PD falls as liquidity rises	25.92 / 32.08 / 36.59	**Pass**
TC-27	Bank factor: 0 bounces with balance above / not above 40,000	+25 / 0	+25 / 0	**Pass**
TC-28	UI: save an application, then reload the page	Applicant still in ledger	11 rows before and after reload	**Pass**
TC-29	UI: ingest sample batch	Two applicants added	11 to 13 rows	**Pass**

Table 8.3 Functional and boundary test cases (executed against the project's service modules)

Summary. Of the 29 cases, 28 behaved as specified (the remaining case, TC-09, is an observation rather than a pass/fail check). The tests confirm that the thresholds in the policy are applied exactly at their boundaries (580 versus 579 for the bureau knockout; 2 versus 3 delinquencies), that disabling a rule removes it from evaluation, that rejected applications receive no limit or tenure, and that the PD is bounded between 0.3% and 88%. The banking cases confirm that two or more cheque bounces trigger the knockout while a single bounce does not, and that bounces and average balance move the champion PD in the expected direction.

8.4.3 Findings from testing

- **Knockout rules carry real weight (TC-09).** With all rules disabled, a prime profile whose bureau score was lowered to 540 was approved, because the other features remain strong and the PD stays below the auto-approve limit. The knockout layer is therefore an essential safeguard, which supports keeping it independent of the model.
- **Seed labels versus engine output.** The seed data tags Vikram Patel as "Manual Review" and Amitabh Das as "Referred (Thin-File)", but under the default champion model they receive PDs of 32.08% and 74.81% and are rejected for excess credit risk. Under the Random Forest they receive 20.1% and 11.11%. The champion model's coefficients are conservative for moderate-risk profiles, and Amitabh's cheque bounce adds further risk (see Section 10.3).
- **Model disagreement is visible.** For preset 2 the three models give 32.08%, 20.10% and 38.50% (XGBoost-style, Random Forest and Logistic Regression), which shows why the model selector is useful for review.
- **Banking terms affect only the champion (TC-25, TC-26).** For preset 2, the champion PD rises from 32.08% to 53.74% as cheque bounces go from 0 to 2, while the Random Forest (20.1%) and Logistic Regression (38.5%) values do not change, so the three models are not compared on identical information for bank-heavy profiles.

CHAPTER 9
Deployment

9.1 Steps to Deploy

1. Install Node.js 18 or later and confirm the installation with node --version and npm --version.
2. Open a terminal in the project folder (finsight-credit-engine) and install dependencies with: npm install
3. Start the development server with: npm run dev. The application opens on http://localhost:3000.
4. Create the production build with: npm run build. This runs the TypeScript compiler (tsc) and then vite build, producing the dist/ folder.
5. Optionally verify the production build locally with: npm run preview.
6. Upload or publish the contents of dist/ to a static host (Section 9.3).
7. Optional: install the Python packages with pip install -r ml_pipeline/requirements.txt and run python ml_pipeline/train_models.py to reproduce the evaluation report.

9.2 Environment Configuration

Item	Configuration
Dev server	vite.config.ts sets port 3000 and does not auto-open a browser.
Build	tsc (strict, ES2020 target, noEmit) followed by vite build; output in dist/.
Styling	tailwind.config.js scans index.html and src/**/*.{js,ts,jsx,tsx}; PostCSS applies Tailwind and Autoprefixer.
Environment variables	None required. No API keys, database URLs or secrets are used.
Network at runtime	Only the Google Fonts stylesheet is fetched; the app works with fallback fonts if offline.
Python pipeline	Independent of the web app; seed 42 makes the synthetic dataset reproducible.

Table 9.1 Environment configuration

9.3 Hosting of Frontend and Backend

The front end is a static bundle and can be hosted on any static service: Netlify or Vercel (connect the repository, set the build command to npm run build and the publish directory to dist), GitHub Pages (publish the dist folder), or a conventional web server such as Nginx or Apache serving dist/ as its document root. Because the single-page app uses tab state rather than URL routes, no special rewrite rules are needed.

There is no separately hosted back end, since the decision engine runs in the browser. If the engine is later moved behind an API, it can be deployed as a Node.js service or as serverless functions behind an API gateway, with the front end calling it over HTTPS (Chapter 11).

CHAPTER 10
Challenges & Limitations

10.1 Issues Faced during Development

- **Making a model decision explainable.** Complex models output a probability but not reasons, whereas lenders must give reasons. The design problem was to attach a consistent, human-readable explanation to every decision.
- **Keeping model, policy and interface consistent.** The PD, score, grade, pricing, rules, cut-offs and limit all depend on one another, and any inconsistency would show up immediately in the interface (for instance, a score that disagrees with a grade).
- **Real-time interactivity.** The underwriter must recalculate the complete pipeline on every slider movement without lag, and the batch table must re-evaluate all rows when data change.
- **Absence of real lending data.** Real, labelled credit data are confidential, so the project had to rely on synthetic and simulated data for development and demonstration.
- **Presenting dense analytics clearly.** Fitting KPIs, charts, tables and decision reasons on screens of different sizes required a disciplined layout, colour semantics and responsive grids.

10.2 Solutions Applied

- Explanations were generated by a dedicated explainer module that returns signed point contributions, ranks and sentences, and are shown both on screen and on the audit certificate.
- All business logic was concentrated in pure functions in a service layer with shared TypeScript types, so that one definition of each ratio, grade and threshold feeds every screen.
- Results are derived with React's useMemo from the inputs rather than stored, which keeps the interface consistent and recalculation instantaneous.
- A seeded synthetic data generator and clearly labelled simulated portfolio data allow the system to be demonstrated reproducibly.
- A single design system (Tailwind tokens, semantic decision colours and a repeated page pattern) keeps the six modules coherent.

10.3 Current Limitations or Known Bugs

- **Calibrated approximations instead of trained models.** The in-browser scorer uses hand-calibrated log-odds functions that imitate a logistic regression, a random forest and an XGBoost model. The benchmark metrics (AUC, KS, precision, recall and F1) and the ROC and KS series are reference values defined in the code and printed by the Python pipeline; the pipeline generates the synthetic dataset but does not fit models with scikit-learn or XGBoost. The headline numbers should therefore be read as design benchmarks, not as measured results on held-out real data.
- **SHAP-style, not computed SHAP.** The factor attributions are rule-based point bands that follow the additive logic of SHAP; they are not Shapley values computed from a fitted model.
- **Champion calibration is conservative.** As shown in Section 8.4, preset applicants labelled "Manual Review" and "Referred" are rejected by the default engine. Their intended label and the engine output should be aligned by recalibrating the coefficients or the presets.
- **Policy edits are not applied to the live underwriter.** The Policy Rules Editor keeps its own state and drives only its impact simulator; the Live Underwriter and Batch Processor use the default rules and cut-offs. The simulator is a heuristic, and the "publish" button only confirms the action.
- **Rule wording versus threshold.** The delinquency knockout is described as "2 or more" in its description, but its threshold (> 2) triggers at three or more; the description and the threshold should be made consistent.
- **Static portfolio data.** Dashboard figures (14,850 applicants, 68% approval rate, 3.1% default rate and the grade distribution) are simulated constants; they are not computed from the ten-applicant batch or from the scoring engine.
- **Browser-only persistence, no authentication or live integrations.** The applicant ledger is saved only in the browser's localStorage, so it is not shared between users or devices and is lost if the browser data is cleared. There is no user management, the certificate's verification hash is a fixed display value, bank-statement fields are typed in rather than verified through bureau, banking or account-aggregator APIs, and PAN and account numbers are stored unencrypted in the browser.
- **Banking terms apply only to the champion model.** The cheque-bounce penalty and liquidity term exist only in the XGBoost-style variant; the Random Forest and Logistic Regression variants ignore bank conduct. The six seed applicants APP-1005 to APP-1010 carry no bank fields, so they default to zero bounces.

- **Type-check error with TypeScript 5.9.** The project has no src/vite-env.d.ts, so tsc reports TS2882 for the side-effect import of ./index.css when the installed TypeScript is 5.9, which makes npm run build fail at the type-check step. Adding a file containing the line /// <reference types="vite/client" /> removes the error (tsc then completes without errors).
- **Fairness not assessed.** No bias or disparate-impact testing has been carried out on the scoring features.

CHAPTER 11
Future Enhancements

11.1 Planned Features

- **Alternative data signals** – incorporate GST invoicing, utility payments and UPI transaction velocity to improve scoring for thin-file and gig-economy applicants.
- **Continuous retraining and drift monitoring** – a retraining pipeline with automated Population Stability Index (PSI) alerts to detect shifts in applicant populations.
- **Core-banking integration** – an API gateway to systems such as Finacle or Temenos for instant disbursal after approval.
- **Account-aggregator bank-statement ingestion** replace the manually entered balance, inflow and cheque-bounce fields with verified data from account-aggregator or bank-statement APIs.
- **True trained models and TreeSHAP** – fit logistic regression, random forest and XGBoost with scikit-learn and XGBoost on real or public credit data, report measured metrics on a hold-out set and replace the rule-based attributions with computed SHAP values.
- **Authentication and role-based access** – analyst, underwriter, policy officer and auditor roles with single sign-on.
- **Policy versioning** – wire the Policy Rules Editor into the live engine, with approval workflow, effective dates and history of every published policy.

11.2 Possible Integrations or Optimizations

- **Back-end service and database** – expose the engine as a REST API (Node.js or Python FastAPI) and replace the browser localStorage ledger by persisting applications, decisions, policy versions and audit records in PostgreSQL, using the entity model in Section 5.2.
- **Signed audit trail** – generate a real SHA-256 digest of each decision record and store it immutably.
- **Fairness and compliance tooling** – disparate-impact testing, adverse-action letter generation and reason-code mapping aligned with lender policy.
- **Expected-loss modelling** – extend from PD to loss-given-default and exposure-at-default for expected credit loss and capital estimation.
- **Automated testing and CI** – convert the test cases of Chapter 8 into a unit-test suite and a Postman collection, and run them on every commit.

- **Performance and accessibility** – code-splitting of the chart libraries, keyboard navigation and screen-reader labels.

CHAPTER 12
Conclusion

12.1 Summary of the Project

FinSight brings together the main elements of a modern credit underwriting system in one web application. An applicant's financial and bureau profile is converted into engineered risk features; a Probability of Default is estimated by a selectable model; the PD is converted to a 300–900 score, a credit grade and a risk-based interest rate; a configurable policy layer of knockouts, flags and cut-offs turns the score into an approve, refer or reject decision with a limit and a reason; and a factor attribution and printable certificate explain the outcome. Around this engine, the application offers a portfolio dashboard, model benchmarking, a policy editor and a bank loan ledger with batch decisioning.

12.2 What Was Achieved

- A working six-module single-page application built with React 18, TypeScript and Tailwind CSS, with a clean separation between interface, decision logic and data.
- A bank-style loan origination flow with KYC and bank-statement intake, a cheque-bounce knockout, banking-conduct scoring and a persistent applicant ledger with CSV export.
- A decision engine in which the PD, score, grade, APR, limit and reason are produced consistently and instantly, and which behaves exactly at its policy boundaries (Chapter 8).
- Three comparable model variants, with the champion showing the best benchmark separation (AUC-ROC 0.884, KS 52.6%) in the project's evaluation matrix.
- Explainability at both decision and portfolio level, and an audit certificate suitable for review.
- A reproducible Python pipeline for synthetic data generation and benchmark reporting, and a complete academic showcase of the work.

Taken together, the project meets the six objectives stated in Section 1.4: a data pipeline, engineered features, comparative models, an automated underwriting engine, explainability and an interactive dashboard. Its limitations (Section 10.3) are clear and give a concrete path to a production-grade system.

12.3 Skills Learned during Development

- Credit risk concepts: Probability of Default, DTI and LTI, bureau scoring, rating grades, risk-based pricing, knockout versus score-based policy and the Basel IRB idea of calibrated PD.
- Machine-learning evaluation: AUC-ROC, KS statistic, precision, recall and F1, champion-challenger comparison and the trade-off between accuracy and interpretability.
- Explainable AI: additive feature attribution in the style of SHAP and writing reason codes for decisions.
- Modern front-end engineering: React hooks, derived state with useMemo, TypeScript domain modelling, Tailwind design systems, Recharts and responsive layout.
- Software engineering practice: layered architecture, pure functions for testability, boundary testing, build tooling with Vite and documenting a system.
- Teamwork across modelling, engineering and explainability roles within a faculty-guided project.

CHAPTER 13
References

[1] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[2] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Advances in Neural Information Processing Systems 30 (NeurIPS), 2017.

[3] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[4] D. W. Hosmer, S. Lemeshow and R. X. Sturdivant, Applied Logistic Regression, 3rd ed. Hoboken, NJ: Wiley, 2013.

[5] N. Siddiqi, Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring. Hoboken, NJ: Wiley, 2006.

[6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[7] Basel Committee on Banking Supervision, "International Convergence of Capital Measurement and Capital Standards: A Revised Framework" (Basel II), Bank for International Settlements, 2006.

[8] Basel Committee on Banking Supervision, "Basel III: A global regulatory framework for more resilient banks and banking systems," Bank for International Settlements, 2011.

[9] Reserve Bank of India, Guidelines on Digital Lending and Fair Practices Code for Lenders, https://www.rbi.org.in.

[10] React documentation, https://react.dev.

[11] TypeScript documentation, https://www.typescriptlang.org/docs/.

[12] Vite documentation, https://vitejs.dev/guide/.

[13] Tailwind CSS documentation, https://tailwindcss.com/docs.

[14] Recharts documentation, https://recharts.org.

[15] XGBoost documentation, https://xgboost.readthedocs.io.

[16] SHAP documentation, https://shap.readthedocs.io.

CHAPTER 14
Appendices

Appendix A – Screenshots of the Application

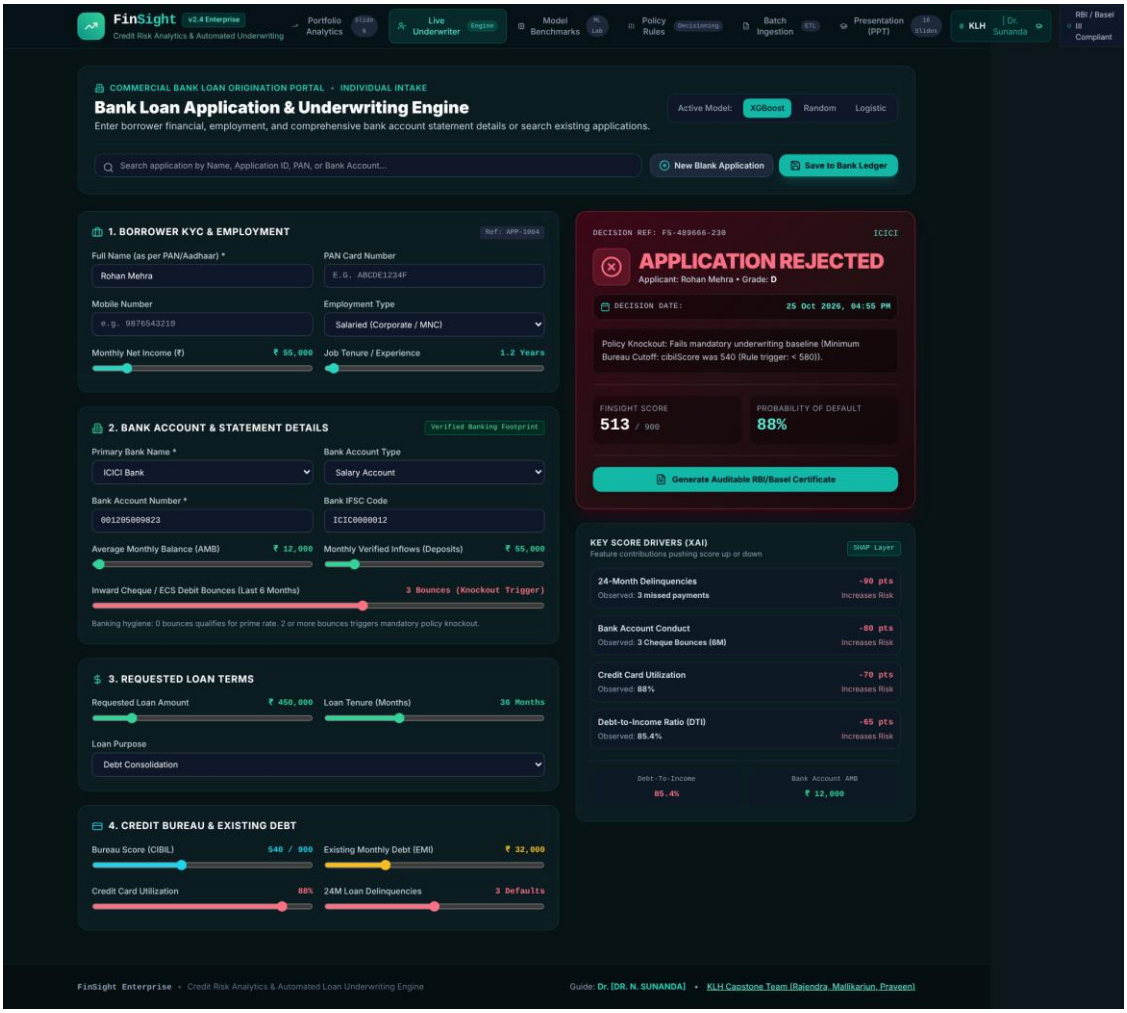


Fig. A.1 Live underwriter: applicant rejected by policy knockouts

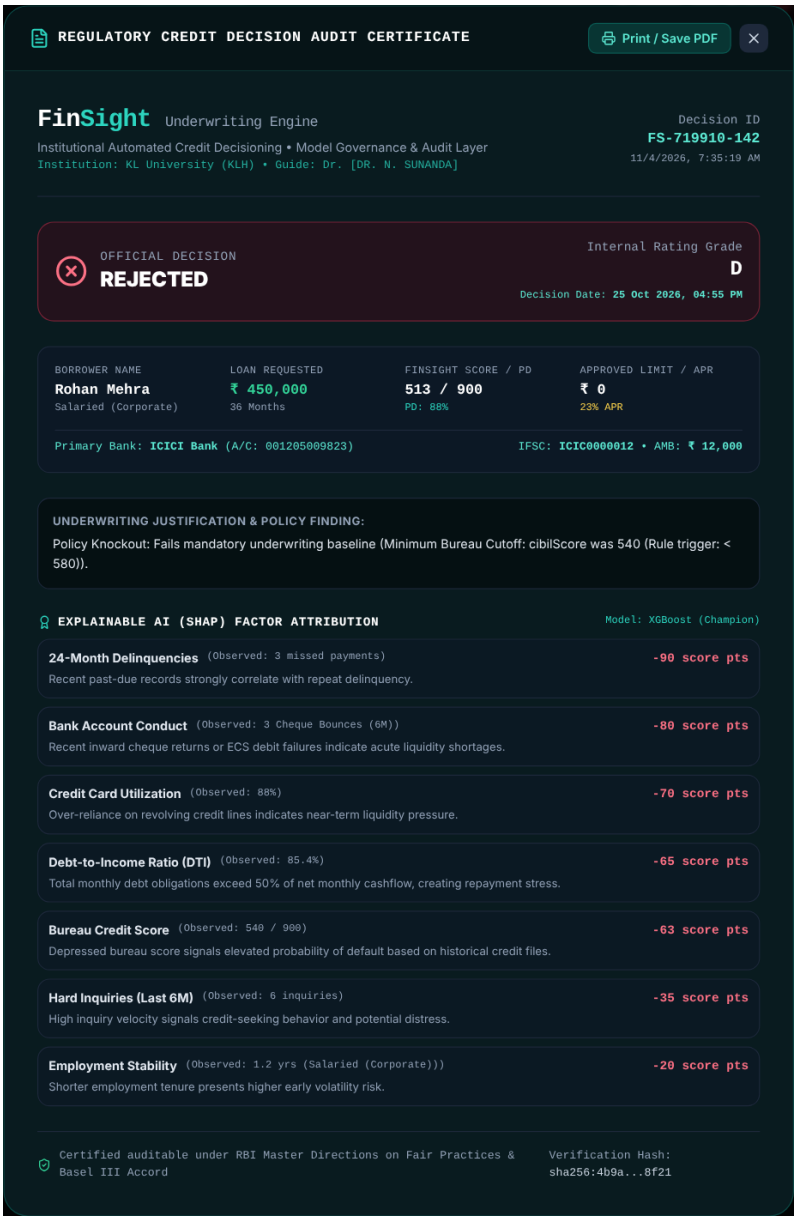


Fig. A.2 Audit certificate of a rejected decision

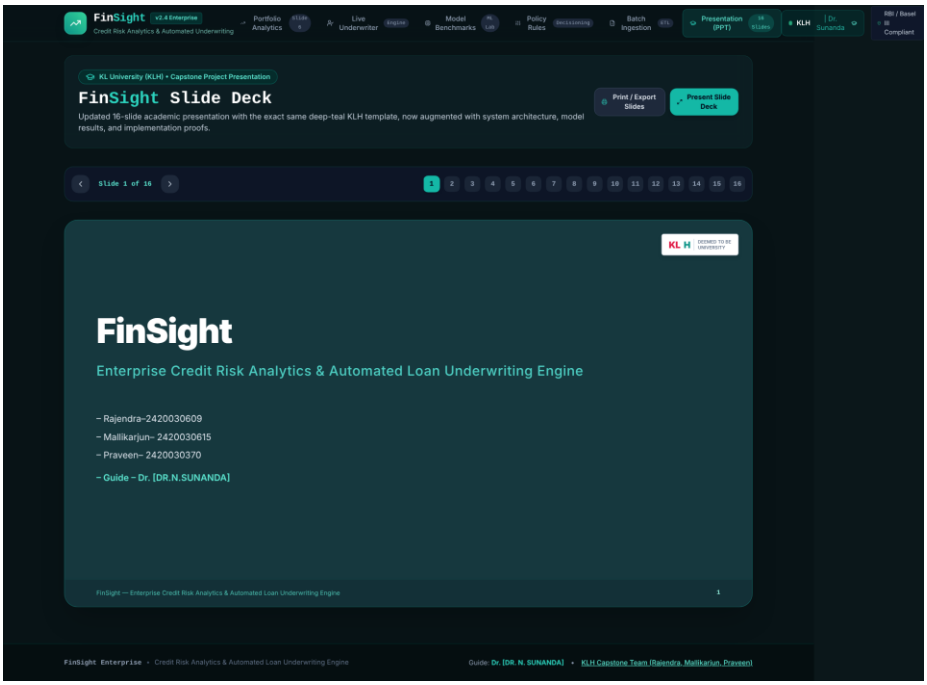


Fig. A.3 Academic showcase (presentation walkthrough)

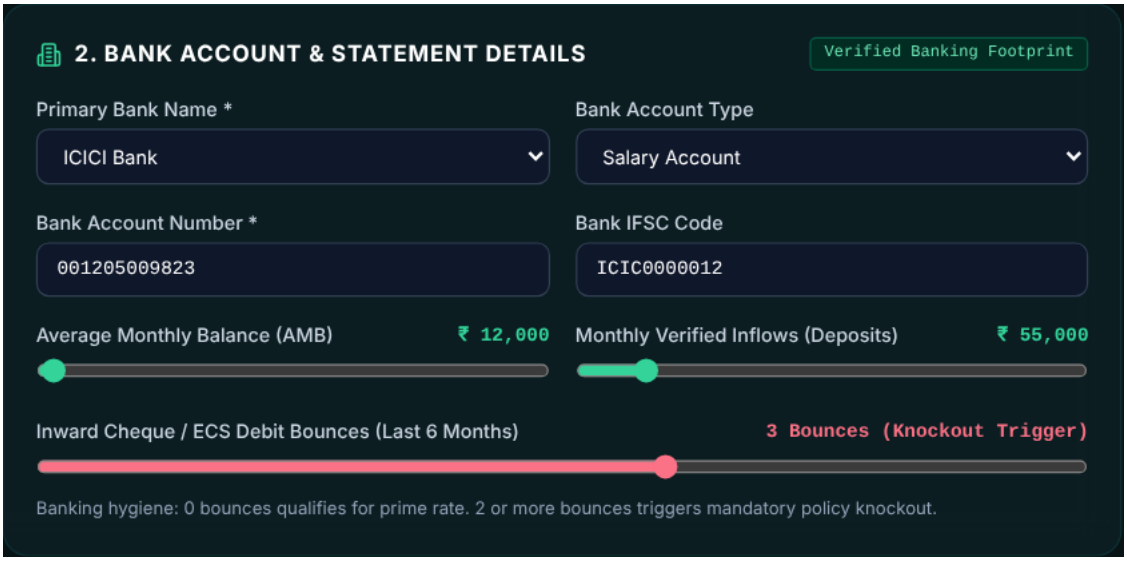


Fig. A.4 Bank account and statement details with the cheque-bounce knockout warning

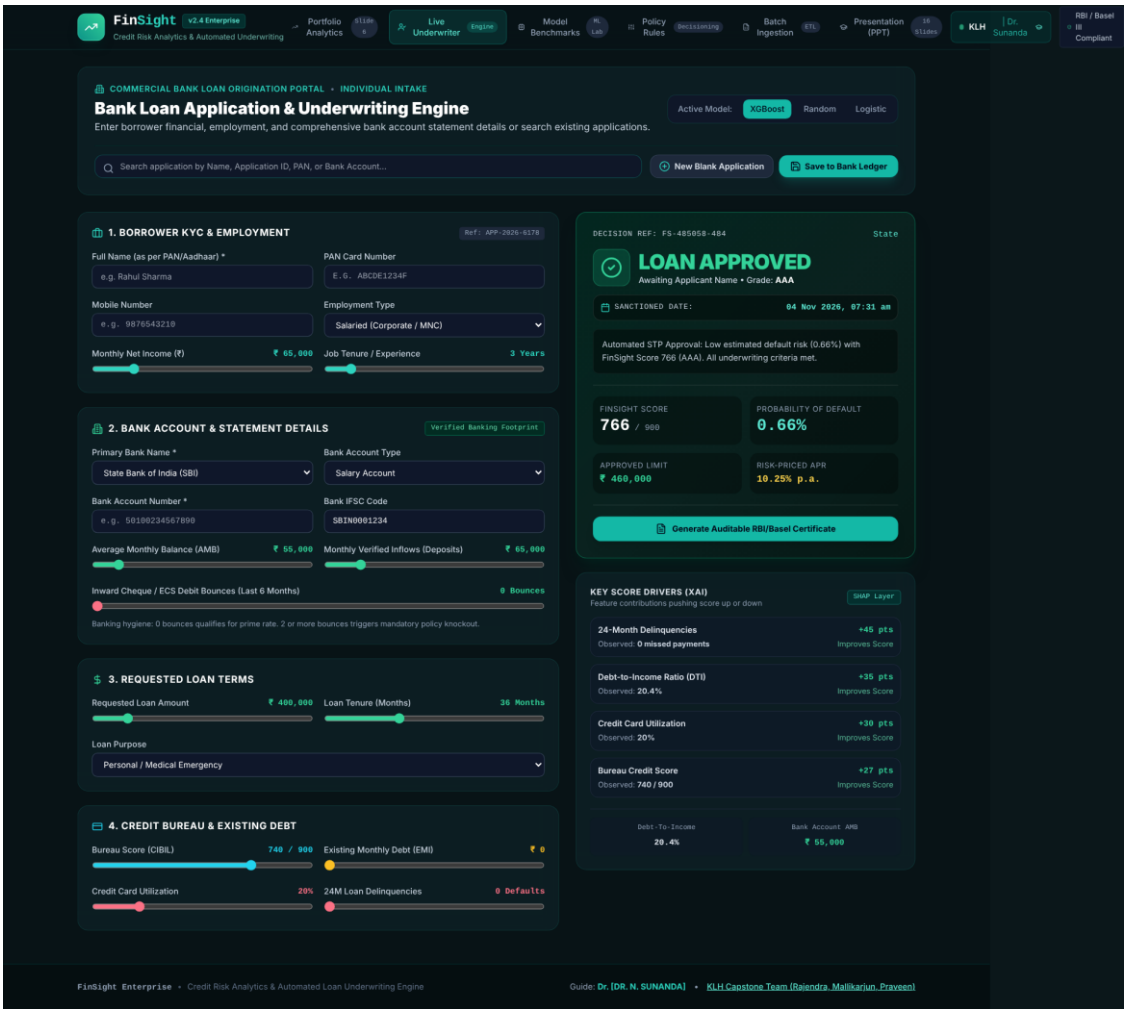


Fig. A.5 Blank loan application (new intake form of the Live Underwriter)

Appendix B – Sample Code Snippets

B.1 Debt-to-Income and Loan-to-Income (scoringEngine.ts)

```
export function calculateDti(applicant: Applicant): number {
  // Approximate EMI using standard reducing balance: P * r * (1+r)^n / ((1+r)^n - 1)
  const monthlyRate = (12.0 / 12) / 100;
  const n = applicant.requestedTenureMonths || 36;
  const newEmiEstimate = (applicant.requestedLoanAmount * monthlyRate * Math.pow(1 + monthlyRate,
n)) /
                         (Math.pow(1 + monthlyRate, n) - 1);

  const totalMonthlyObligation = applicant.existingMonthlyEmi + newEmiEstimate;
  const dti = (totalMonthlyObligation / Math.max(applicant.monthlyNetIncome, 1000)) * 100;
  return Number(dti.toFixed(1));
}

/**
 * Calculates Loan to Annual Income ratio
 */
export function calculateLti(applicant: Applicant): number {
  const annualIncome = applicant.monthlyNetIncome * 12;
  return Number((applicant.requestedLoanAmount / Math.max(annualIncome, 10000)).toFixed(2));
}
```

B.2 Probability of Default – champion model branch (scoringEngine.ts)

```
    // XGBoost: Gradient boosted with cross-feature interaction penalties
    const interactionPenalty = (dti > 45 && applicant.creditCardUtilization > 65) ? 0.75 : 0;
```

```
    const severeDelinqPenalty = (applicant.delinquenciesLast24Months >= 2) ? 1.6 :
(applicant.delinquenciesLast24Months === 1 ? 0.6 : -0.4);
    const superPrimeDiscount = (applicant.cibilScore > 780 && dti < 30) ? -0.8 : 0;

    logit = -3.1
      - (1.05 * normBureau)
      + (0.75 * normDti)
      + (0.65 * normUtil)
      + severeDelinqPenalty
      + (0.38 * normInq)
      - (0.42 * normTenure)
      + (0.30 * lti)
      + interactionPenalty
      + superPrimeDiscount
+ employmentBonus
+ bouncePenalty
+ liquidityDiscount;
  }
  // Sigmoid transfer to probability: P = 1 / (1 + exp(-logit))
  const rawProb = 1 / (1 + Math.exp(-logit));
  // Bound between 0.3% and 88.0%
  const boundedProb = Math.min(Math.max(rawProb, 0.003), 0.88);
  return Number((boundedProb * 100).toFixed(2));
}
```

B.3 Credit grade and risk-based pricing (scoringEngine.ts)

```
export function getCreditGrade(pdPercent: number): UnderwritingResult['creditGrade'] {
  if (pdPercent < 1.5) return 'AAA';
  if (pdPercent < 3.0) return 'AA';
  if (pdPercent < 5.0) return 'A';
  if (pdPercent < 8.0) return 'BBB';
  if (pdPercent < 12.0) return 'BB';
  if (pdPercent < 18.0) return 'B';
  if (pdPercent < 28.0) return 'CCC';
  return 'D';
}

/**
 * Calculates risk-based interest rate (APR)
 */
export function calculateRiskBasedInterestRate(grade: UnderwritingResult['creditGrade']): number
{
  const margins: Record<UnderwritingResult['creditGrade'], number> = {
    'AAA': 1.75, // 10.25%
    'AA': 2.50,  // 11.00%
    'A': 3.50,   // 12.00%
    'BBB': 4.75, // 13.25%
    'BB': 6.50,  // 15.00%
    'B': 8.50,   // 17.00%
    'CCC': 11.00,// 19.50%
    'D': 14.50   // 23.00%
  };
  return Number((BASE_INTEREST_RATE + margins[grade]).toFixed(2));
}
```

B.4 Policy rule evaluation (underwritingPolicy.ts)

```
  // Check custom rules
  for (const rule of rules) {
    if (!rule.enabled) continue;

    let val = 0;
    if (rule.field === 'dti') val = dti;
    else if (rule.field in applicant) {
      val = (applicant as any)[rule.field];
    }

    let triggered = false;
    if (rule.operator === '<' && val < rule.threshold) triggered = true;
    else if (rule.operator === '>' && val > rule.threshold) triggered = true;
    else if (rule.operator === '<=' && val <= rule.threshold) triggered = true;
```

```
    else if (rule.operator === '>=' && val >= rule.threshold) triggered = true;
    else if (rule.operator === '==' && val === rule.threshold) triggered = true;

    if (triggered) {
      if (rule.category === 'KNOCKOUT') {
        knockoutsTriggered.push(`${rule.name}: ${rule.field} was ${val} (Rule trigger:
${rule.operator} ${rule.threshold})`);
      } else {
        softFlagsTriggered.push(`${rule.name}: ${rule.field} was ${val}`);
      }
    }
  }
```

B.5 Limit calculation (underwritingPolicy.ts)

```
  // Max Approved Limit calculation
  let maxApprovedLimit = applicant.requestedLoanAmount;
  if (decision === 'REJECTED') {
    maxApprovedLimit = 0;
  } else if (decision === 'REFERRED') {
    // Recommend conservative counter-offer of 70% of requested or capped by income
    maxApprovedLimit = Math.min(applicant.requestedLoanAmount * 0.75, applicant.monthlyNetIncome
* 12);
  } else {
    // Approved: Can qualify for up to 120% if prime
    maxApprovedLimit = creditGrade === 'AAA' || creditGrade === 'AA'
      ? Math.round(applicant.requestedLoanAmount * 1.15 / 10000) * 10000
      : applicant.requestedLoanAmount;
  }
```

B.6 Factor attribution – delinquency factor (shapExplainer.ts)

```
  // 3. Delinquency Track Record (24 Mo)
  let delinqImpact = 0;
  if (applicant.delinquenciesLast24Months === 0) delinqImpact = +45;
  else if (applicant.delinquenciesLast24Months === 1) delinqImpact = -40;
  else delinqImpact = -90;
  factors.push({
    featureKey: 'delinquenciesLast24Months',
    featureName: '24-Month Delinquencies',
    actualValue: `${applicant.delinquenciesLast24Months} missed payments`,
    contributionScore: delinqImpact,
    riskDirection: delinqImpact >= 0 ? 'RISK_DECREASING' : 'RISK_INCREASING',
    importanceRank: 3,
    explanation: applicant.delinquenciesLast24Months === 0
      ? `Spotless 24-month repayment record indicates superior credit discipline.`
      : `Recent past-due records strongly correlate with repeat delinquency.`
  });
```

B.7 Synthetic data and latent risk function (train_models.py)

```
    # Synthetic credit applicant distributions
    cibil_score = np.random.normal(680, 75, n_samples).clip(300, 900)
    monthly_income = np.random.lognormal(11.2, 0.6, n_samples).clip(15000, 500000)
    existing_emi = monthly_income * np.random.beta(2, 5, n_samples)
    loan_requested = np.random.uniform(50000, 1500000, n_samples)
    tenure_months = np.random.choice([12, 24, 36, 48, 60], n_samples)
    card_utilization = np.random.beta(2, 3, n_samples) * 100
    delinquencies_24m = np.random.poisson(0.3, n_samples).clip(0, 5)
    inquiries_6m = np.random.poisson(1.2, n_samples).clip(0, 10)
    tenure_years = np.random.gamma(2.5, 1.8, n_samples).clip(0.5, 30)

    print("[Step 2/5] Engineering risk features (DTI, LTI, Repayment History, Capacity)...")
    # Estimated monthly EMI
    monthly_rate = 0.12 / 12
    new_emi = (loan_requested * monthly_rate * ((1 + monthly_rate)**tenure_months)) / (((1 +
monthly_rate)**tenure_months) - 1)
    dti = ((existing_emi + new_emi) / monthly_income) * 100
    lti = loan_requested / (monthly_income * 12)

    # True latent default probability generator
    logit = (
        -3.2
```

```
        - 0.012 * (cibil_score - 650)
        + 0.045 * (dti - 35)
        + 0.025 * (card_utilization - 30)
        + 0.850 * delinquencies_24m
        + 0.220 * inquiries_6m
        - 0.080 * tenure_years
        + 0.350 * lti
    )
    p_default = 1 / (1 + np.exp(-logit))
    default_actual = (np.random.uniform(0, 1, n_samples) < p_default).astype(int)
```

Appendix C – Installation / Setup Instructions

Step	Command / action	Result
1	Install Node.js (v18 or later) from nodejs.org	node and npm available on the command line
2	Extract finsight-credit-engine.zip and open a terminal in the extracted folder	Project folder ready
3	npm install	Dependencies from package.json installed in node_modules
4	npm run dev	Development server at http://localhost:3000
5	npm run build	Type-checked production bundle in dist/
6	npm run preview	Local preview of the production build
7	pip install -r ml_pipeline/requirements.txt	Python packages installed (optional)
8	python ml_pipeline/train_models.py	Synthetic dataset generated and evaluation matrix printed (optional)

Table C.1 Setup steps

Appendix D – User Manual

D.1 Reviewing portfolio risk

1. Open the application; the Portfolio Analytics tab is shown by default.
2. Read the three KPI cards, the decision mix chart and the quarterly default-rate trend.
3. Use the quarter and loan-type filters to narrow the view, and review the grade distribution and ageing buckets.

D.2 Underwriting a single applicant

1. Open Live Underwriter and either search for a saved applicant (name, ID, PAN or bank account) or complete the blank form: borrower KYC, bank-statement details, loan terms and bureau fields.
2. Choose the model (XGBoost champion, Random Forest or Logistic Regression).
3. Read the decision banner, score, grade, PD, APR and limit on the right, and review the SHAP contributions.
4. Select the certificate button to open the audit certificate; use Print to save it as a PDF.
5. Select Save to Bank Ledger to keep the application; it then appears in the Batch Ingestion tab.

D.3 Comparing models

1. Open Model Benchmarks, compare the three model cards and switch between ROC, KS and feature-importance views.

D.4 Tuning policy

1. Open Policy Rules, toggle rules on or off and move the threshold and cut-off sliders.
2. Read the impact simulator to see the expected approval rate, default rate and STP efficiency; use Reset to return to defaults.

D.5 Running a batch

1. Open Batch Ingestion, review the pipeline stages and the ledger table.
2. Search by name, ID, bank, account number or date and filter by decision; use the ingest action to add new applicants.
3. Select the export button to download the filtered decisions as a CSV file; use the row action to open a certificate.

Appendix E – Certifications

[Attach scanned copies of any course, internship or workshop certificates earned by the team members in relation to this project here.]